

系统程序设计基础——大抄

考核情况

- ❖ 实验情况 (40~50%)
 - ❧ 常规作业
 - ❧ 机试
 - ❧ 免实验: CSP200/ICPC、CCPC省级奖项/CACC初赛奖项
- ❖ 考勤 (扣分项: 10%)
- ❖ 课程ppt更正 (加分项)
- ❖ 期末考试 (50~60%)

比例会动态调整

课程背景与目标

- ❖ 课程背景
 - ❧ 承上启下, 编程学习不间断
 - ❧ 培养学生设计、实现并优化复杂系统程序的专业能力
- ❖ 课程目标
 - ❧ 巩固程序设计技术
 - ❧ 培养模块化设计与工程化思维
 - ❧ 深入理解与运用指针及内存管理
 - ❧ 提升程序性能优化与底层编程能力

教学团队



教师: 吴锋
wufeng02@ustc.edu.cn



教师: 徐伟
xuwei hf@ustc.edu.cn



助教: 储李节
cljjkk
@mail.ustc.edu.cn



助教: 夏玉杰
chieh
@mail.ustc.edu.cn



助教: 王宇航
wyh2022
@mail.ustc.edu.cn

程序设计课程回顾与UML简介

徐伟

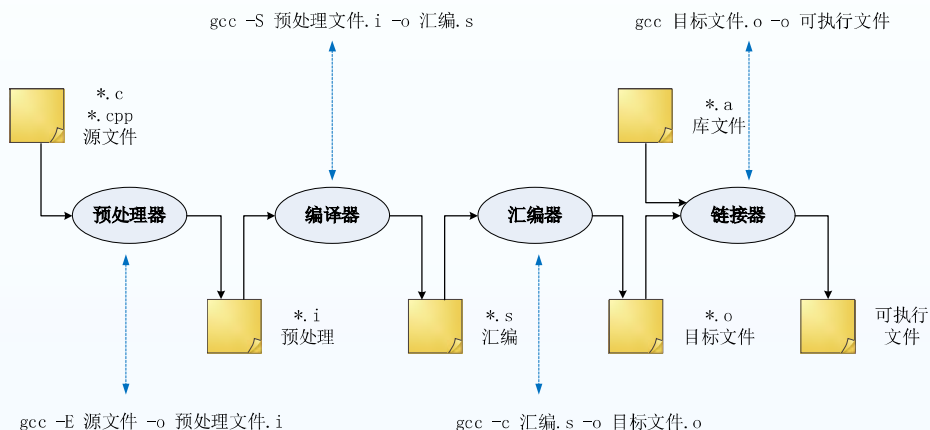
E-mail: xuweihf@ustc.edu.cn

程序的开发与编译

gcc/g++选项	功能	目标文件常用后缀
-E (大写)	预处理指定的源文件，不进行编译。	.i
-S (大写)	编译指定的源文件，生成 汇编文件，但是不进行汇编。	.s
-c	编译、汇编指定的源文件，生成目标文件，但是不进行链接。	.o
-o	指定生成目标文件的文件名	
-m32	按照32位进行编译	
-m64	按照64位进行编译	

有关更多编译指令，可自行查看[GCC手册](#)

程序的开发与编译

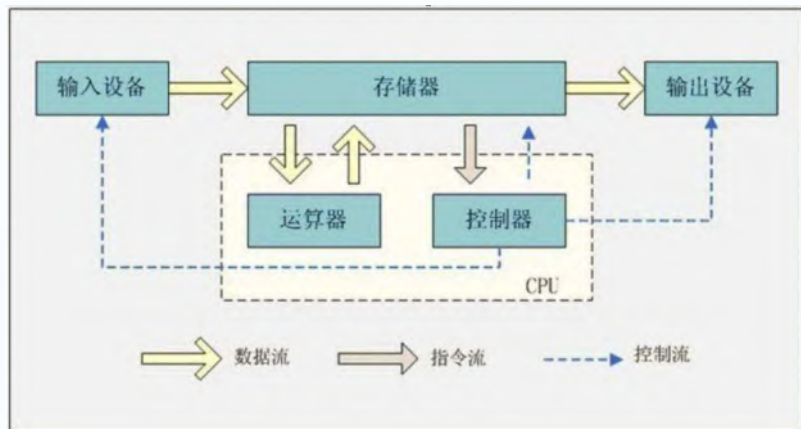


程序的开发与编译

优化等级	简要说明
-Ofast	在-O3级别的基础上，开启更多 激进优化项 ，该优化等级不会严格遵循语言标准
-O3	在-O2级别的基础上，开启了更多的高级优化项，以编译时间、代码大小、内存为代价获取更高的性能。
-Os	在-O2级别的基础上，开启 降低生成代码体量的优化
-O2	开启了大多数 中级优化 ，会改善编译时间开销和最终生成代码性能
-O/-O1	优化效果介于-O0和-O2之间
-O0	默认优化等级，即 不开启编译优化 ，只尝试减少编译时间

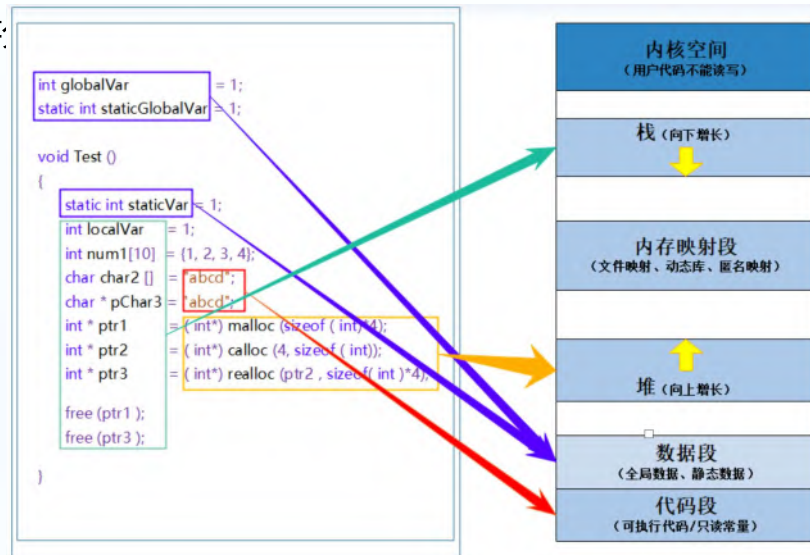
程序是如何运行起来的

❖ 计算机



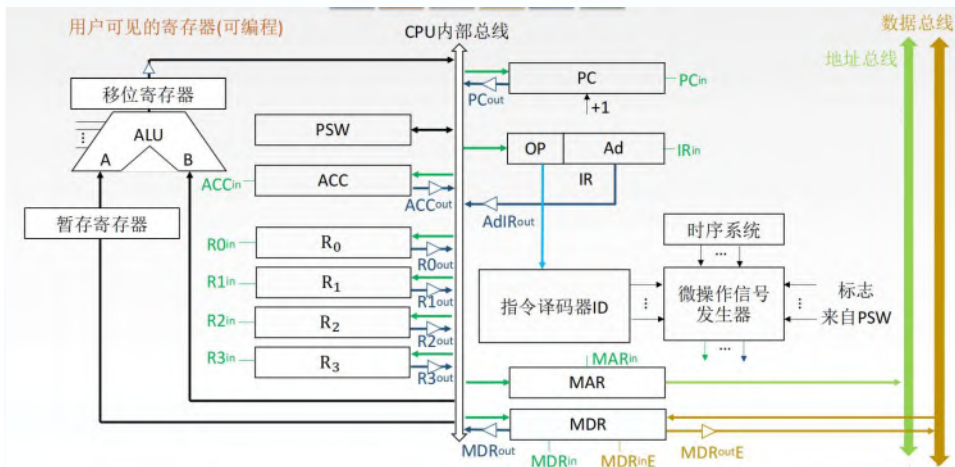
程序是如何运行起来的

❖ 内存:



程序是如何运行起来的

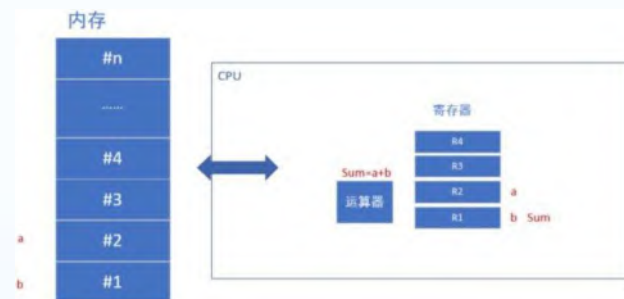
❖ CPU结构



程序是如何运行起来的

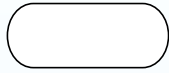
❖ CPU和内存的交互

```
#include <stdio.h>
int main ()
{
    int a = 5, b = 3, sum;
    sum = a + b;
    printf("%d\n", sum);
    return 0;
}
```



流程图

❖ 基本符号



开始/
结束



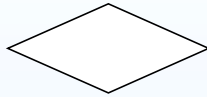
数据输入
输出



处理



准备或
预处理



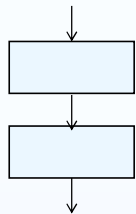
条件判断



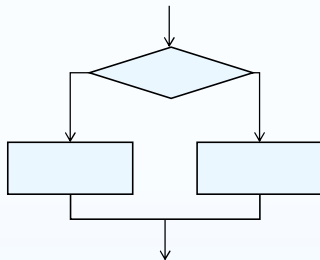
文件或
文档

流程图

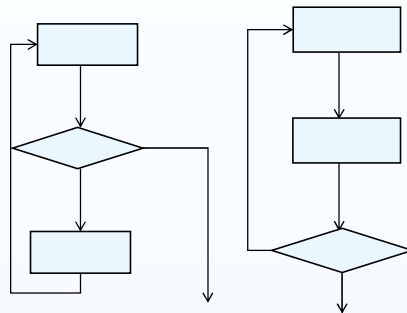
❖ 基本结构



顺序



分支



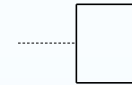
循环

流程图

❖ 基本符号



流线



注解或注
释

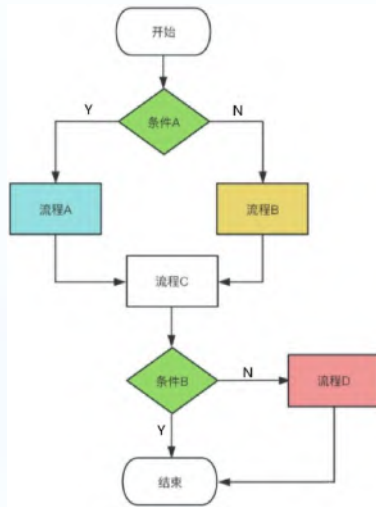
流程图

❖ 注意:

- ❧ 请使用标准的框图符号，而不是自定义符号
- ❧ 至少包括开始、结束和一个处理
- ❧ 整体绘制方向是从上到下、从左往右
- ❧ 除了判断框外，大多数框图只有一个流入和一个流出
- ❧ 开始框只有一个流出线，终止框只有一个流入线
- ❧ 判断框输出线需要标注判断条件
- ❧ 线与线避免交叉
- ❧ 语言尽可能精炼

流程图

❖ 示例



UML

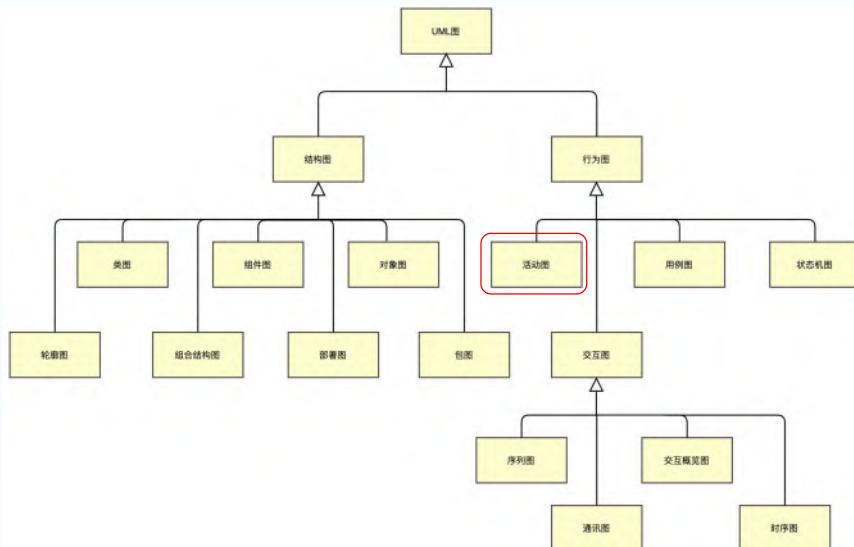
❖ UML

- ❧ Unified Model Language
- ❧ 中文：统一建模语言
- ❧ 是由一整套图表组成的标准化建模语言

❖ 为什么要UML

- ❧ 对整个软件设计有更好的可读性，可理解性，从而降低开发风险
- ❧ 方便各个开发人员之间的交流

UML



活动图

- ❖ UML活动图以图形化的方式展示**控制流**和**对象流**
- ❖ 强调从一个活动到另一个活动的顺序、分支和并发
- ❖ 常用于描述内部的工作流程、业务过程或操作的详细逻辑
- ❖ 与流程图的区别
 - ❧ 并发建模
 - ❖ 通过分叉和汇合节点直接支持并发流程，传统流程图通常只能描述单一线程
 - ❧ 责任划分（引入泳道）
 - ❧ 面向对象（与用例图、类图衔接）

活动图

❖ 基本符号

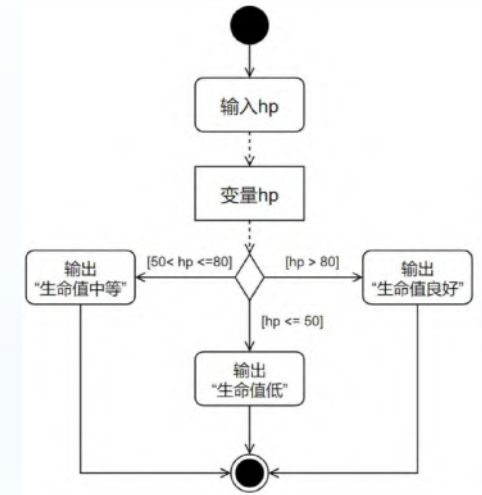


活动图

❖ 分支结构

```
#include <stdio.h>
```

```
void main() {
    int hp;
    scanf("%d", &hp);
    if(hp > 80)
        printf("生命值良好");
    else if(hp > 50)
        printf("生命值中等");
    else
        printf("生命值低");
}
```



分支不仅仅能够表示二元条件，也可以表示多元条件判断

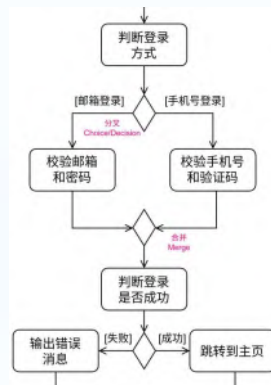
活动图

❖ 步骤合并

菱形也表示多个步骤合并为一条路径，或者多个变量输入到一个步骤的过程

```
...
if(a == 1)
    bVal = true;
else if(b == 2)
    bVal = true;

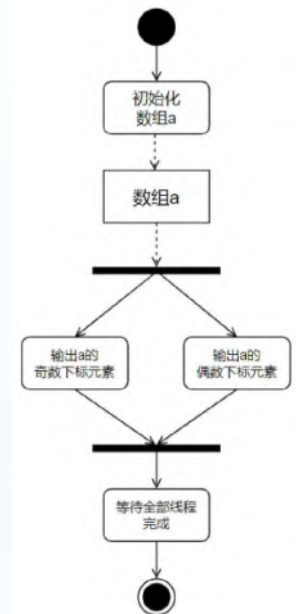
If(bVal)
    callFun();
else
    printf("fail");
...
```



活动图

❖ 并行与汇合

```
...
线程1读取数组a的奇数位
线程2读取数组a的偶数位
读取完毕后，输出数组a
...
```

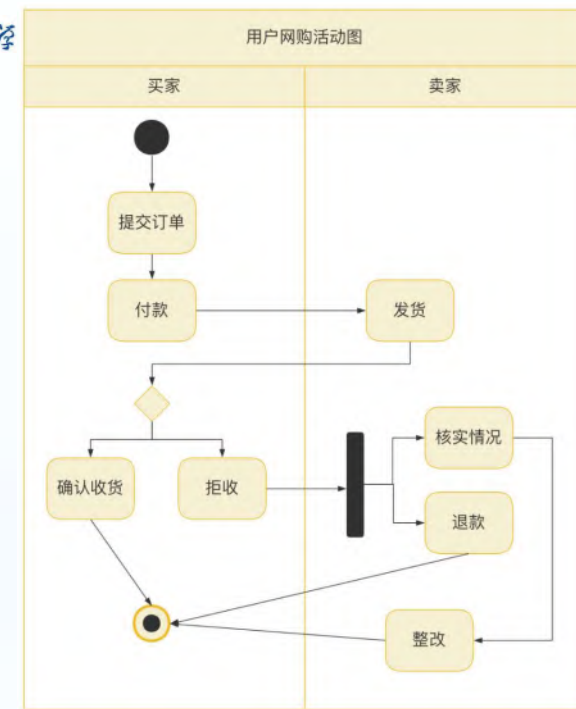


活动图

❖ 泳道

- 清楚的表达这些活动或动作是由谁来完成的
- 每个泳道都以唯一的对象的名称或活动者的名称来命名
- 活动或动作位于泳道内，不可以跨越泳道，而活动的转移可以跨越泳道

❖ 泳道

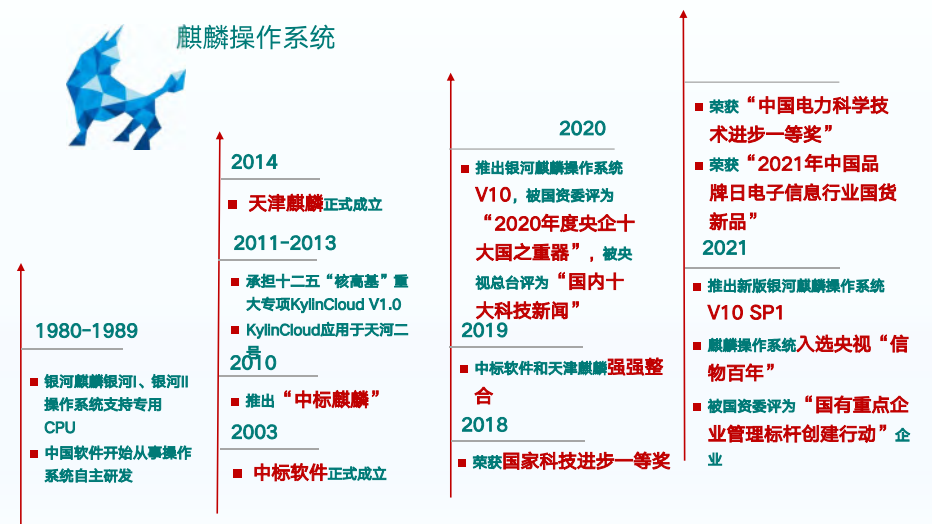


系统程序设计基础：Linux简介1

徐伟

E-mail: xuweihf@ustc.edu.cn

国产Linux发行版



Linux软件安装与卸载

- ❖ 软件安装需要有root权限 (sudo/su)

❖ 源码包

❖ 脚本安装包

❖ 二进制包

❖ 在线安装

❖ Ubuntu

apt-get update

apt-get install gcc 安装

apt-get remove gcc 卸载

❖ Centos

yum install gcc

yum remove gcc

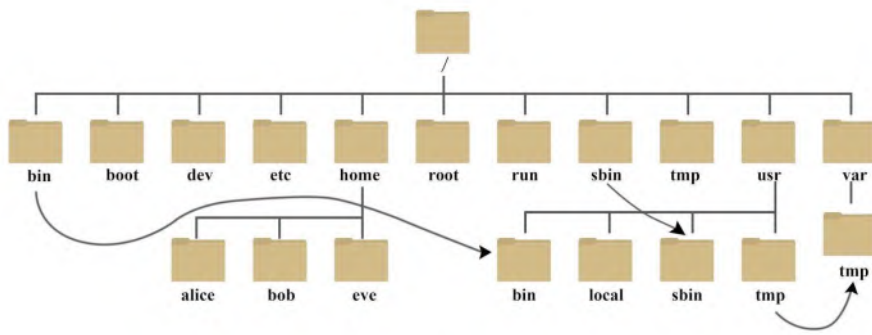
20

键盘快捷方式

- ❖ Ctrl-c: 停止命令执行
- ❖ Ctrl-d: 结束传输或屏幕输入
- ❖ Ctrl-s: 临时停止输出
- ❖ Ctrl-q: 恢复输出
- ❖ Ctrl-u: 擦除光标以前的
- ❖ Ctrl-k: 擦除光标以后的
- ❖ backspace: 纠正光标前字符
- ❖ Ctrl-r : 在以前的命令中搜索

22

Linux文件系统



❖ HOME 用户文件夹

❖ Bin - 二进制可执行程序。ls、cp、bash...

❖ Boot - 启动文件

❖ Dev - 设备文件夹, 如硬盘sda

❖ ...

24

文件的类型

- ❖ 利用ls -l输出对应文件名的行第一个字符可判断
- ❖ 普通文件
- ❖ 目录文件: 以d开头, 如:
 - ❖ drwxr-xr-x 2 root root 4096 Aug 2 2006 bin

```
xxxwww@ubuntu:~/Desktop/compiler/2021fall-compiler_cminus-private$ ls -l
total 36
drwxrwxr-x 5 xxxwww xxxwww 4096 May 30 00:00 build
-rw-rw-r-- 1 xxxwww xxxwww 800 May 28 16:29 CMakeLists.txt
drwxrwxr-x 4 xxxwww xxxwww 4096 May 28 16:29 include
-rw-rw-r-- 1 xxxwww xxxwww 1240 May 28 16:29 README.md
drwxrwxr-x 6 xxxwww xxxwww 4096 May 28 16:29 Reports
-rw-rw-r-- 1 xxxwww xxxwww 350 May 28 16:29 shell.nix
drwxrwxr-x 8 xxxwww xxxwww 4096 May 28 16:29 src
drwxrwxr-x 6 xxxwww xxxwww 4096 May 28 16:29 tests
xxxwww@ubuntu:~/Desktop/compiler/2021fall-compiler_cminus-private$
```

27

文件的类型

- ❖ 利用ls -l输出对应文件名的行第一个字符可判断
- ❖ 普通文件
- ❖ 目录文件
- ❖ 设备文件

❖ 块设备文件：以b开头

brw-rw----1 root disk 3, 1 Jan 30 2003 /dev/hda1

❖ 字符设备文件：以c开头

crw-----1 root root 4, 1 Jul 31 13:49 /dev/tty1

```
xxxwww@ubuntu:/dev$ ls -l
total 0
crw-r--r-- 1 root root 10, 235 Oct 7 19:19 autofs
drwxr-xr-x 2 root root 540 Oct 7 19:20 block
drwxr-xr-x 2 root root 80 Oct 7 19:19 bsg
crw----- 1 root root 10, 234 Oct 7 19:19 btrfs-control
drwxr-xr-x 3 root root 60 Oct 7 19:19 bus
lrwxrwxrwx 1 root root 3 Oct 7 19:19 cdrom -> sr0
lrwxrwxrwx 1 root root 3 Oct 7 19:19 cdrw -> sr0
drwxr-xr-x 2 root root 3700 Oct 7 19:20 char
crw--w---- 1 root tty 5, 1 Oct 7 19:19 console
```

文件的类型

- ❖ 利用ls -l输出对应文件名的行第一个字符可判断
- ❖ 普通文件
- ❖ 目录文件
- ❖ 设备文件
- ❖ 链接文件

❖ 存放文件系统通向文件的路径，以l开头

```
xxxwww@ubuntu:/dev$ ls -l
total 0
crw-r--r-- 1 root root 10, 235 Oct 7 19:19 autofs
drwxr-xr-x 2 root root 540 Oct 7 19:20 block
drwxr-xr-x 2 root root 80 Oct 7 19:19 bsg
crw----- 1 root root 10, 234 Oct 7 19:19 btrfs-control
drwxr-xr-x 3 root root 60 Oct 7 19:19 bus
lrwxrwxrwx 1 root root 3 Oct 7 19:19 cdrom -> sr0
lrwxrwxrwx 1 root root 3 Oct 7 19:19 cdrw -> sr0
drwxr-xr-x 2 root root 3700 Oct 7 19:20 char
crw--w---- 1 root tty 5, 1 Oct 7 19:19 console
```

29

目录操作命令

- ❖ ls : 显示目录中的内容
- ❖ pwd: 显示当前工作目录
- ❖ cd: 改变用户工作目录
- ❖ mkdir: 建立用户目录
- ❖ rmdir: 删除目录

其他常用命令

- ❖ mv : 移动文件
- ❖ cat : 显示文件内容
- ❖ chmod : 改变文件权限
- ❖ sudo/su : 以系统管理者的身份执行指令
- ❖ poweroff : 关机
- ❖ reboot : 重启

模块化程序设计

徐 伟

E-mail: xuweihf@ustc.edu.cn

单文件与多文件开发

❖ 单文件

- ❏ 代码全在一个文件
- ❏ 快速原型验证
- ❏ 个人项目足够



❖ 多文件

- ❏ 代码复用
- ❏ 独立测试
- ❏ 迭代演进
- ❏ 避免冲突

核心问题

如何把一个大程序拆成多个小模块
使其易维护、可复用、可测试？

模块化编程概念与原则

❖ 模块

❏ 硬件模块

- ❖ 计算机的某一种或几种特定功能，做成一个独立的、可以插拔的硬件部件
- ❖ 内存、硬盘、CPU、显卡...

❏ 软件模块

- ❖ 将复杂的软件程序，根据功能，拆分成一个个独立的、可以互相协作的**代码块**
- ❖ 每个模块负责一项具体的任务，对外隐藏了内部的实现细节，只留下一些“**接口**”供其他模块使用
- ❖ 函数、类、库...

模块化设计三原则

❖ 高内聚 (High Cohesion)

- ❏ 同一模块只做一类事，功能单一、职责明确

❖ 低耦合 (Low Coupling)

- ❏ 模块间依赖最少、依赖方向稳定

❖ 信息隐藏 (Information Hiding)

- ❏ 对外只承诺接口，对内可自由更换实现

模块边界如何定

- ❖ 把经常变化的东西隔离起来
 - ❏ 配置、平台差异、算法选择、 I/O
- ❖ 把可复用的能力抽成模块
 - ❏ 容器（vector/map）、字符串处理、日志、错误处理
- ❖ 让依赖方向“朝稳定处走”
 - ❏ 业务依赖接口，不依赖具体实现文件
- ❖ 经验法则
 - ❏ 模块对外只暴露最少必须的信息（类型名/函数/常量）

小技巧

先写 .h，再写 .c：逼自己先想清楚“别人怎么用”。

写 .h 之前先做接口设计

- ❖ 接口五问
 - ❏ 这个模块提供什么能力？（最小集合）
 - ❏ 输入/输出是什么？返回值是否足够表达错误？
 - ❏ 资源/所有权怎么交接？谁负责 create/destroy？
 - ❏ 哪些内容必须暴露为类型？哪些可以只暴露“句柄”（不透明指针）？
 - ❏ 依赖哪些别的模块？能不能只依赖“抽象”（前向声明/回调）？

模块化设计

- ❖ 不好的模块化设计
 - ❏ 改一个结构体字段，很多文件都要改
 - ❏ 需要“到处加 include”才能编译
 - ❏ 新增功能要改很多旧代码（回归风险高）
- ❖ 好的模块化设计
 - ❏ 改动集中在少数文件
 - ❏ 依赖链短、方向清晰
 - ❏ 新增功能主要是“加代码”不是“改代码”



前向声明与回调

	前向声明	回调
核心作用	提前告知编译器某个类型/函数存在，允许引用而不需完整定义	将可执行代码作为参数传递，实现反向调用和定制行为
目的	解决依赖、隐藏实现、减少编译耦合	解耦调用者与被调用者，增强灵活性
常见场景	句柄类型声明、相互递归的结构体、接口头文件设计	事件处理、异步操作、算法定制（如排序比较器）

思考题

❖ 如果把所有结构体细节都放到.h, 会发生什么?

- ❧ 封装性破坏
- ❧ 编译依赖爆炸, 构建变慢
- ❧ 兼容性丧失
- ❧ 安全风险
- ❧ ...

如何减少重新编译

❖ 使用前向声明

- ❧ 减少头文件中的include

❖ 信息隐藏

- ❧ 实现细节放到.c, 只在.h中暴露必要的接口

❖ 合理组织模块

- ❧ 避免大而全的头文件

推荐的头文件组织

Project/		规则
include/	#对外API (安装/发布)	▸ include/: 只放稳定接口 ▸ src/internal/: 实现细节可以随时改 ▸ 公共头文件不 include 私有头
vec.h		
logger.h		
src #实现		
vec.c		
logger.c		
internal/	#私有头 (仅src内部使用)	▸ 依赖方向清晰 ▸ 更易信息隐藏 ▸ 编译更快 (减少include)
vec_impl.h		
tests/	#测试/示例 (可选)	
test_vec.c		

头文件模板

```
// vec.h (公共接口)
#ifndef VEC_H
#define VEC_H

#include <stddef.h>    // 只include自己真的需要的

typedef struct Vec Vec; // 不透明指针: 隐藏实现细节

Vec* vec_create(void);
void vec_destroy(Vec* v);
int  vec_push(Vec* v, int x); // 返回错误码
size_t vec_size(const Vec* v);

#endif // VEC_H
```

#ifndef/#define: 包含保护, 防止头文件被重复包含导致重复定义

.h与.c职责边界

.h文件：对外接口

- ▶ 类型定义 (typedef, struct)
- ▶ 宏常量 (#define)
- ▶ 函数声明
- ▶ 必要的注释说明

.c文件：内部实现细节

- ▶ 私有函数
- ▶ 私有数据
- ▶ 算法与优化
- ▶ 内部实现

只暴露 ”用得上的声明”，不暴露 ”能暴露的一切”

自治

❖ 一个头文件是自治 (Self-Contained) 的，说明：

- ☞ 它独自被#include时能够编译通过，不依赖其他头文件被提前包含
- ☞ 它显式地包含了直接使用的所有类型、宏、函数的声明

反例：隐式依赖

```
// vec.h
void vec_add(Vector* v, size_t n); // 使用了 size_t, 但没有包含 <stddef.h>
```

若某个.c文件

```
#include "vec.h" // 编译错误! size_t 未定义
```

必须改为

```
#include <stddef.h>
```

```
#include "vec.h" // 正确, 但依赖包含顺序
```

“必须先包含 A 才能包含 B”的依赖就是不自治

头文件设计原则

❖ 包含保护

☞ #ifndef/#define/#endif 或 #pragma once

❖ 自治

❖ 依赖最小化

❖ 禁止循环包含

❖ 避免变量定义

❖ 避免函数实现

❖ 模块合理命名

❖ ...

自治

❖ 自治 (Self-Contained) 重要性

☞ 避免隐式顺序约束

❖ 使用者无需记住头文件的包含顺序，降低出错概率。

☞ 提高可维护性

❖ 修改头文件时，不会因为依赖关系隐藏而破坏现有代码。

☞ 简化代码审查和重构

❖ 每个头文件都是独立的单元，易于理解和测试

依赖最小化

❖ 采用前向声明等方式，减少依赖

- ☞ 只声明类型名，不包含完整定义
- ☞ 减少依赖，只用指针时不需要#include

```
// a.h
typedef struct A A;    // 前向声明
typedef struct B B;    // 前向声明

void a_do_something(A* a, B* b); // 只使用指针，不需要完整定义
```

如何避免循环包含

- ❖ 使用前向声明（Forward Declaration）代替包含
- ❖ 将互相依赖的类型放到同一个头文件中
- ❖ 检查循环包含的工具
 - ☞ 头文件依赖检查软件（如include-graph等）
- ❖ 使用#pragma once
 - ☞ 不能解决设计上的循环
 - ☞ 可以防止同一文件多次包含，配合前向声明使用更简洁

禁止循环包含

❖ 循环包含

- ☞ 头文件 A 直接或间接地包含了头文件 B，而头文件 B 又直接或间接地包含了头文件 A

```
// a.h
#include "b.h"
```

```
// b.h
#include "a.h"
```

❖ 循环包含的问题

- ☞ 编译错误
- ☞ 设计耦合
- ☞ 难以维护

避免变量定义

❖ 除非static const或extern声明

- ☞ static变量具有内部链接，即只在定义它的编译单元（当前.c文件及其直接或间接包含的头文件）内可见
- ☞ extern声明变量而不定义，告诉编译器变量在别处定义

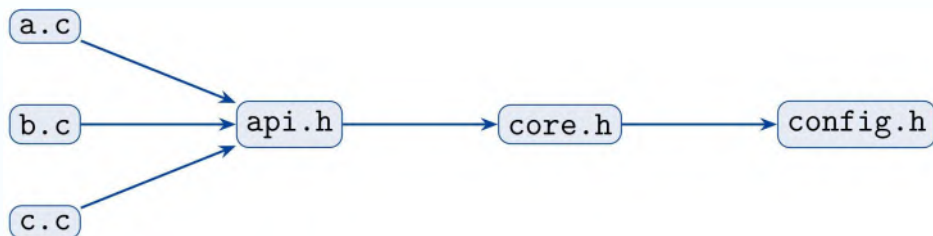
```
// counter.h
extern int g_count;    // 只是声明，不分配内存
```

```
// counter.c
int g_count = 0;       // 真正的定义，分配内存
```

避免函数实现

- ❖ 当头文件被多个 .c 文件包含时，导致多重定义
 - ☞ 链接时，链接器发现多个相同的符号，会报“重复定义”错误
- ❖ 编译时间增加
- ❖ 代码膨胀
- ❖ 破坏封装

重编译风暴的消除



- ❖ #include 是文本替换，会带来传递依赖
- ❖ 头文件越“胖”，重编译范围越大
- ❖ 头文件规范（前向声明/最小包含）能显著减少耦合

模块合理命名

- ❖ 避免不通模块的函数名冲突（C语言没有命名空间）
- ❖ 同一模块的所有函数用统一前缀（如 vec_*、logger_*）

例：

```
vec_create()
vec_push()
vec_size()
```

Makefile基础

- ❖ 变量
 - ☞ 变量一般定义在文件头部
 - ☞ 使用\$(变量名)表示变量

符号	含义
=	赋值
+=	增加赋值
:=	常量赋值

```

1  TAR = test      #赋值
2  CC := gcc       #常量赋值
3
4  $(TAR):test.o   #常量使用
5  $(CC) test.o -o $(TAR)
6  test.o:test.s
7  $(CC) -c test.s -o test.o
8  test.s:test.i
9  $(CC) -s test.i -o test.s
10 test.i:test.c
11 $(CC) -E test.c -o test.i
    
```

Makefile基础-通配符

通配符	含义
%	模式字符，用来通配任意个字符
\$@	目标文件名。多目标模式规则中，它代表的是触发规则被执行的文件名
\$%	当目标文件是一个静态库文件时，代表静态库的一个成员名
\$<	第一个依赖的文件名
\$?	所有比目标文件更新的依赖文件列表
^	代表的是所有依赖文件列表
+	保留了依赖文件中重复出现的文件。主要用在程序链接时库的交叉引用场合

什么是CMake

- ❖ 不同平台编译所使用的Makefile标准、格式不同
- ❖ CMake编写一种与平台无关的文件来指导整个编译流程，根据目标平台生成所需要的Makefile和工程文件
- ❖ CMake具有最小的依赖性，只需要在工作系统上安装编译器即可

Makefile基础

❖ 条件判断

If ifeq ifneq ifdef ifndef

以endif结尾

复杂条件判断使用elif和else

Example:

```
libs_for_gcc = -lgnu
normal_libs =
ifeq ($(CC),gcc)
    libs=$(libs_for_gcc)
else
    libs=$(normal_libs)
endif
```

多文件编译错误排查



多文件编译错误排查

错误信息	常见原因	优先排查
undefined reference	漏编译/漏链接；库顺序错误；声明与定义不一致	目标文件是否参与链接；-L/-l是否正确；符号名是否拼写一致
multiple definition	在头文件放了定义；同名全局变量/函数重复定义	是否把变量定义写进 .h；是否缺少extern；是否重复链接了同一对象文件

目标文件内容

部分	含义
.text	机器指令（函数代码）
.data	已初始化的全局/静态变量
.bss	未初始化的全局/静态变量（占位，不占文件体积）
符号表	“我提供哪些符号 / 我还缺哪些符号”

链接器做的事

把多个 .o 拼起来：符号解析 + 地址重定位

多文件编译错误排查

❖ 现象

undefined reference to ...

nm 列出目标文件（.o 文件、静态库 .a、可执行文件）的符号表，每个符号前有一个字母表示其类型

多文件编译错误排查

❖ 现象

undefined reference to ...

rg (ripgrep) 是超快的代码搜索工具，用来在源码目录中查找符号的定义或引用

相当于“更现代、更好用、更快的 grep -R”，专门为代码仓库搜索优化。

rg 是“为代码仓库量身定制的 grep”。

指针的高级应用

吴锋

Email: wufeng02@ustc.edu.cn

指针的安全使用原则

- ❖ 指针必须初始化: `int *p = NULL;`
- ❖ 使用前检查有效性: `if (p != NULL) *p = 10;`
- ❖ 释放后置为 `NULL`: `free(p); p = NULL;`
- ❖ 避免类型不匹配: `int *p_int = p_char; // 尽量避免`
- ❖ 谨慎使用类型转换: `int *p_int = (int *)p_void; // 谨慎使用`
- ❖ 擅用 `const` 修饰符修饰指针:

```
// 指针指向的内容不可修改
const int *p = NULL; int num = 10;

p = &num; // 允许修改指针所指向的变量
// *p = 20; // 错误: 不可修改指向的值
```

指向常量的指针

```
// 指针本身不可修改
int *const p = &num; // 初始化指针

*p = 20; // 允许修改指针指向变量的值
// p = &other; // 错误: 不可修改指针指向
```

指针常量

指针变量的声明

❖ 指向复杂数据类型的指针

```
❖ int (*ptr)[5]; // ptr是指向由5个整型元素构成的数组的指针变量
❖ int *parr[5]; // parr是一个5个整型指针元素构成的指针数组
❖ int **ip; // ip是一个“指向整型量的指针变量”的指针变量
❖ int *fip(); // fip是返回整型指针的函数（函数声明）
❖ int (*pti)(); // pti是指向一个返回整数的函数的指针变量
❖ int *(*pfpi)(); // pfpi是一个指向函数的指针变量，该函数返回整数指针
```

❖ 指针变量也占据内存空间（在32/64位系统上分别占4/8字节）

❖ 调试指针时可使用 `%p` 打印指针所指向的地址

```
int num = 10;
int *p = &num;
printf("Address of num: %p\n", (void *)p);
// 输出地址如: 0x7fffa1c9818c
```

指向数组的指针

- ❖ 声明变量时的数组名是一个符号地址常量，可以理解为指针常量，指向数组的首元素地址，是右值，不能对数组名赋值
- ❖ 除了数组名的值不能改变外，数组名和指针在很多地方可以通用，例如 `int a[5], *iptr = a;`

```
❖ iptr = a; 等价于 iptr = &a[0]; 即取元素地址
❖ a[3] ⇔ *(a+3) ⇔ *(iptr+3) ⇔ iptr[3]
❖ *(iptr+3) 的运算过程: 取出指针变量 iptr 的值, 加上 3×sizeof(int), 得到新地址 &a[3], 再取值得到内容 a[3]
❖ a[3] 的运算过程: 取出数组首地址, 加上 3×sizeof(int), 得到新地址 &a[3], 再取值得到内容 a[3]
```

指向二维数组的指针

❖ 定义形式：类型名 (*变量名)[数组大小]

如：int (*pa)[4], pa 是指向“由4个整型量构成的数组”的指针变量

❖ 若有 int a[2][4], 则可以 pa = a;

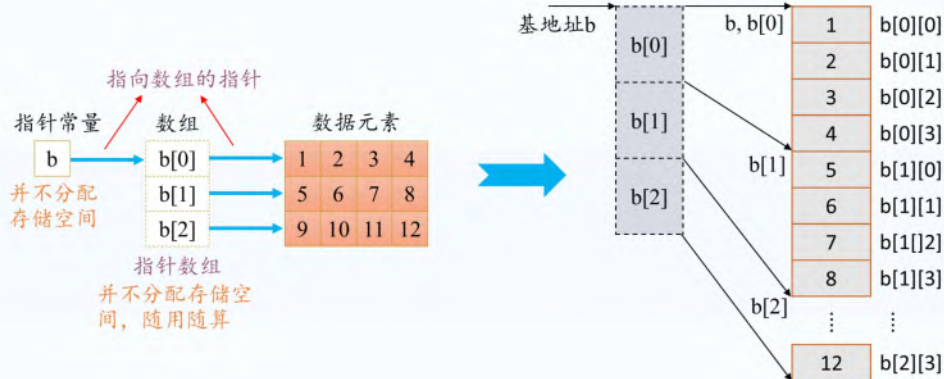
如：char (*next)[16], next 是指向“由16个字符构成的数组”的指针变量

❖ 若有 char n[5][16], 则可以 next = n;

指向多维数组的指针

❖ 多维数组虽然也是连续存储在内存中的元素集合，但语法上（编程使用时）是元素为数组的数组

❖ 例如：int b[3][4]={1,2,3,4}, {5,6,7,8}, {9,10,11,12};



指向二维数组的指针

❖ 数组取值时，每处理一个[]或*, 相当于一次指针取值运算

如 int a[3][4];

如 a, a+i, 指向“由4个整型数构成的数组”的指针

如 a[i], a[i]+j, 指向整型数的指针, 即 *(a+i), *(a+i)+j

如 a[i][j], 整型数, 即 (*(a+i)+j) 或 (*(a+i*4+j))

❖ 数组取地址时, &表示指向该类型元素的指针

如 int a[3][4];

如 &a[i][j], 指向整型数的指针, 值为 *(a+i)+j 或 a[i]+j

如 &a[i], 指向“由4个整型数构成的数组”的指针, 值为 a+i

如 &a, 指向二维数组的指针, 值为 a

❖ 注意 &a 与 &a[0] 的差异

❖ ~~a[0]~~ = a[0] = *(a + 0), 二者值相等且类型一样

❖ 思考: 可以有 &(a+0) 吗?

```
printf("%p", &(a+0));
// error: lvalue required as
// unary '&' operand
```

指向字符串常量的指针

❖ 可以定义一个字符指针变量, 让其指向一个字符串常量

例如: const char *msg = "Hello"; // 推荐写法

msg[0] = 'h'; // 错误: 试图修改字符串常量

实际上就是把字符串常量在内存中的首地址赋给该指针

其地位等价于指向一维字符数组第一个元素的指针

注意这个指针指向常量区, 指针值可以改, 指针指向的内容不可以改

❖ 字符数组与字符串常量的差别

字符数组存放在程序活动的栈中, 可以修改

字符串常数存放在常量区, 不可以修改

指向字符串常量的指针

- ❖ 可以用字符串常量对字符指针变量和字符数组初始化
- ❖ 可以用字符串常量对字符指针变量赋值
- ❖ 不可以用字符串常量对数组名赋值（试图修改数组名常量）

```
char *pstr1 = "Hello", str1[40] = "Okay";
char *pstr2, str2[40];
pstr2 = "world";
str2 = "world"; //错误！
```

- ❖ 只能逐个元素地对数组赋值（比如for循环等方法）

指针数组

- ❖ 可以定义由指针作为元素的数组，具体的声明形式：

类型说明符 *数组名[数组长度]

例：int *p[3]; char *name[]=...;

// 注意 int (*p)[3] 与 int *p[3] 的区别

- ❖ 指针数组中的元素都是指向同一类型的指针，常见的应用：
 - ⌚ 若有大量数据（例如学生信息）需要按某个元素排序时（例如按姓名拼音顺序），使用指针数组排序，就可以避免大量数据的搬移
 - ⌚ 当需要对多个已存在的对象进行批量处理时，也可以使用指针数据
 - ❖ 例：游戏程序中各自独立行动的人物（不使用多线程），每人一个指针组成数组，就可以在一个循环中顺序处理所有人物
 - ❖ 例：多线程程序使用一个指针数组/链表来顺序处理各个进程

指针数组

- ❖ 指针数组常用于处理一组字符串，比使用二维数组更方便，也节省内存空间，在排序等操作中效率更高

- ⌚ 例：char *name[]={“allen”, “word”, “jones”, “martin”};
- ⌚ 字符串常量 “allen” 等又称为字符串字面量，它们被存储于常量区
- ⌚ 在代码中，字符串字面量可以等同于一个 char * 指针
 - ❖ 甚至允许 “allen”[3] = ‘e’（虽然编译可以通过，但是执行会发生错误，即常量区不允许改写）
- ⌚ 在这里，相当于在初始化器中列举了一系列字符指针，它们分别指向常数区的字符串

二级指针的安全使用原则

- ❖ 初始化规则：二级指针必须指向有效的一级指针地址
 - ⌚ 例：int **p2; **p2 = 10;
 - // 错误：p2未初始化，指向不确定地址
- ❖ 双重检查：解引用前检查两级指针有效性
 - ⌚ 例：if (p2 != NULL && *p2 != NULL) { **p2 = 10; }
- ❖ 内存释放顺序：先释放底层内存，再释放上层内存
 - ⌚ 例：free(matrix); // 使用后仅释放行指针
 - // 错误：列内存未释放，导致内存泄漏
- ❖ 类型一致性：确保二级指针类型与一级指针类型完全匹配
 - ⌚ 例：int *p1; char **p2 = (char **)&p1;
 - // 错误：类型不匹配，可能导致访问错误

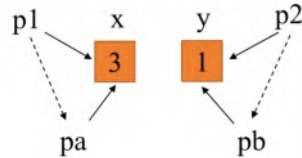
带指针参数的函数

- ❖ 当指针作为函数的参数时，所传的是值，其内容是地址（也称传地址调用）

- ❏ 被调函数通过指针访问函数外部的数据——具有数据双向传递的功效

```
#include<stdio.h>
void swap2(int *pa, int *pb) {
    int t;
    t=*pa; *pa=*pb; *pb=t;
}

void main() {
    int x=1, y=3, *p1=&x, *p2=&y;
    swap2(p1, p2);
    printf("%d,%d,%d,%d\n", x, y, *p1, *p2);
}
```



可变参数函数（例一）

- ❖ VA_LIST宏定义的例子：在任意个整数参数中找出最大值

```
#include <stdarg.h>

int max_int(int n, ...) { //...表示参数可变
    va_list ap;           //定义参数列表，实质是一个指针
    int i, current, largest;
    va_start(ap, n);      //初始化，指出可变长度部分开始的位置
    largest=va_arg(ap, int); //取回单个变量
    for (i=1; i<n; i++) {
        current=va_arg(ap, int); //取回单个变量
        if (current>largest) largest=current;
    }
    va_end(ap);           //结束获取
    return largest;
}

max_int(3, 28, 35, 13); // 返回 35
max_int(5, 28, 35, 13, 65, 42); // 返回 65
```

带指针参数的函数

- ❖ 有时传指针的目的不是为了让函数修改原变量，而是为了节省传递的数据量（避免值拷贝），实际只是读取变量的值

- ❏ 用const来保护指针所指向的变量不被修改
- ❏ 例如：void f(const T *p); 函数内不得修改p所指向对象的值，有时称p是指向常量的指针，常用于T是一个大结构体传值的场景
- ❏ 对比：void f(T * const p); 是指函数内不得修改指针p的指向，但对p指向的对象没有约束，既可修改该对象的值
- ❏ 对比：void f(const T * const p); 在函数内指针p的指向和指针p所指向对象的值都不得修改

- ❖ 帮助编译器检查函数内部是否有非法的（设计时禁止）操作

可变参数函数（例二）

- ❖ VA_LIST宏定义的例子：自定义打印接口的实现

```
int my_printf(const char *fmt, ...) { //...表示参数可变
    va_list args;                    //定义参数列表，实质是一个指针
    va_start(args, fmt);             //初始化，指出可变长度部分开始的位置

    //使用vsprintf一次性读取
    static char gc_PrintfOut[1000];
    vsprintf((char *) gc_PrintfOut, 1000, (char *) fmt, args);
    va_end(args); //结束获取

    puts("%s", (const char *)gc_PrintfOut); //使用
    return 0;
}
```

自定义函数 my_printf("Hello %d", 2026) 的使用方式与 printf 相同

命令行参数的实现

- ❖ 给 main 函数添加参数，可以实现命令行参数

```
int main(int argc, char *argv[]) { }
```

```
int main(int argc, char **argv) { }
```

- ❖ argc 代表传递给main的形参的个数，其值至少为1
- ❖ argv 是指向字符串的指针数组。第一个字符串是程序执行文件名本身，第二个开始是真实参数
- ❖ 这两个参数由操作系统传递给 main 函数

❖ 例: > gcc -o myprograme main.c

函数指针

- ❖ 函数语句存储在一片连续的内存单元中（代码区），其首地址就是函数的入口地址，函数名则代表该地址
- ❖ 主函数在调用子函数时，就是让程序从主函数转移到子函数的入口地址去执行
- ❖ 程序也可以自定义指针，指向函数的入口地址，即指向函数，以此实现函数调用
- ❖ 定义形式：类型 (*指针变量名)(参数表);
 - ❖ 其中类型即函数返回值类型，参数表即函数形参
 - ❖ 如: float (*func)(int, char, float); 注意与 float *func(int x, char y, float z); 的区别

命令行参数的实现（例）

- ❖ 例：命令行参数获取

```
#include <stdio.h>
void main(int argc, char *argv[]) {
    int i = 1;
    while (i < argc)
        printf("%s%c", argv[i++], (i + 1 < argc ? ' ' : '\n'));
}

#include <stdio.h>
void main(int argc, char **argv) {
    int i = 1;
    while (i < argc)
        printf("%s%c", *(++argv), (i + 1 < argc ? ' ' : '\n'));
}
```

./a.exe -o main main.c
// 输出 -o main main.c

函数指针

- ❖ 和任何一个指针一样，指向函数的指针也必须先初始化（指向某个函数），然后才能使用
- ❖ 如: double square(double x); //函数声明
double (*p)(double x); //函数型指针声明
p=square; //指针初始化
s=p(1.6); 或 s=(*p)(1.6); //函数调用
- ❖ 指向的函数的返回值类型和形参必须与指针变量中的定义相一致
- ❖ 整个函数如果需要作为参数在其它函数间进行传送，必须使用指向函数的指针变量作为参数
- ❖ 函数指针的主要作用是编写通用程序

函数指针（例一）

- ❖ 例：两个整数的四则运算（回调机制，动态调用）

```
int add(int a, int b) {
    return a + b;
}
int( * func_ptr)(int, int) = add; // 定义函数指针
printf("%d\n", func_ptr(3, 5)); // 输出8

// 作为函数形参
void process(int a, int b, int( * cb)(int, int)) {
    printf("Result: %d\n", cb(a, b)); // 实现回调机制 (callback)
}
process(4, 6, add); // 输出Result: 10

typedef int( * MathFunc)(int, int);
MathFunc operations[] = { // 定义数组
    add, subtract, multiply, divide };
int result = operations[0](10, 5); // 动态调用, 调用add函数
```

动态内存分配

- ❖ 申请空间：内存分配函数

```
void *calloc(size_t nmemb, size_t size);
```

- ☞ 为nmemb个元素的数组分配连续的内存空间，其中每个元素的长度都是size个字节
- ☞ 分配成功返回指向空类型void的指针，并将所有字节初始化为0；失败返回NULL

- ❖ 分配空间后，可以用 realloc 调整大小

```
void *realloc(void *ptr, size_t size);
```

- ☞ 为ptr指向的动态分配内存重新分配为大小为size个字节的空间。新尺寸可能会大于或小于原有尺寸
- ☞ 分配成功返回指向空类型void的指针，失败返回NULL
- ☞ ptr必须是通过malloc或calloc获得指针

动态内存分配

- ❖ 动态内存分配，是在运行时动态申请一段内存空间，并通过指针的类型转换将其视为一个特定对象来使用

- ☞ 常用于创建动态数组、数据结构的结点

- ❖ 申请空间：内存分配函数

```
#include <stdlib.h>
void *malloc(unsigned int size);
```

- ☞ 分配成功返回指向空类型void的指针，失败返回NULL

- ❖ 常用调用方法（类型转换、大小计算）

```
int *p; p=(int *) malloc(n*sizeof(int));
if (p == NULL) exit(-1); // 处理分配失败情况
```

unsigned int的大小是四字节，最大可申请 2^{32} 字节/4GB空间

动态内存释放

- ❖ 释放空间：动态分配的内存存在使用完毕后应及时释放

```
#include <stdlib.h>
void free(void *pb);
```

- ❖ 常用调用方法

```
free(p);
p=NULL; // 防止野指针，养成好习惯
```

- ❖ 注意：

- ☞ free只能释放malloc分配的内存，不能对随便一个指针都应用free
- ☞ 首地址保存好，否则内存无法释放，将造成内存泄漏

柔性数组与指针成员的区别

特性	柔性数组	指针成员
内存布局	紧跟结构体，连续内存	结构体仅存地址，内存不连续
分配次数	1次 (malloc)	2次 (malloc 结构体 + malloc 数组)
释放次数	1次 (free)	2次 (free 数组 + free 结构体)
访问效率	高 (直接访问)	低 (一次访存获取地址，再访存获取数据)

注意事项

- 柔性数组属于C99标准，在旧的编译器中可能需要使用 [0] 或 [1] 形式。
- 不能直接定义结构体数组变量包含柔性成员(如 `struct Node d[10];`)，因为编译器无法确定总体内存大小。

内存池管理

- ❖ 动态内存分配：在标准 C 中，`malloc` 是通过系统调用（如 `brk` 或 `mmap`）向操作系统申请内存的，在高性能应用中存在以下问题：
 - ❏ 性能开销：频繁的系统调用会导致用户态和内核态之间的切换，浪费 CPU 周期，系统调用的开销在每秒万次级的操作下会变得极其沉重
 - ❏ 内存碎片：频繁分配和释放大小不一的内存块，会导致物理内存中出现大量无法利用的小空隙，最终无法申请到连续的大块内存
 - ❏ 内存泄漏风险：生命周期管理的疏忽会导致程序占用内存持续攀升
- ❖ 内存池管理：预先申请一大块连续内存，然后在程序内部自行管理这块内存的分配和回收，用以满足后续所有的内存需求
- ❖ 内存池的两种主流模型：
 - ❏ 固定大小内存池：预先划分为大小相等的块，效率最高，逻辑最简单。
 - ❏ 变长内存池：允许分配不同大小的内存块，实现较复杂

建立动态数组

❖ 建立一维数组

- ❏ 定义指向所需数据类型的**指针变量**
- ❏ 按照指定的长度和数据类型**申请**分配存储空间
- ❏ 将动态分配获取的存储区域首地址**强制类型转换**后赋值给指针变量

❏ 例如：

```
int *pb;
if ((pb=(int *)malloc(n*sizeof(int)))==NULL) {
    printf("内存不足\n");
    exit(1);
}
```

- ❏ 使用方法与一般指针相同，也可作为一维数组名使用

内存池管理

❖ 内存池的优缺点

维度	内存池	标准 malloc
分配速度	极快 (O(1))	较慢 (搜索链表)
内存碎片	极少 (块固定)	严重 (产生大量空洞)
管理复杂度	较高 (需手动维护池)	简单 (系统托管)
适用场景	高频、固定大小分配	随机、不确定大小分配

❖ 内存池的核心步骤：

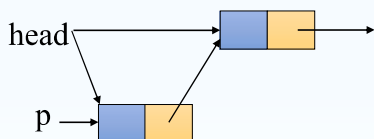
- ❏ 初始化：申请一大块内存，并将其切割成一个个相同大小的块，并串起来
- ❏ 分配：从内存池头部取出一个块，并将头指针指向下一个空闲的内存块
- ❏ 释放：将用完的块插回内存池的头部（操作时间复杂度为 O(1)）

链表的创建

- ❖ 头插法：在链表头顺序插入新结点来建立链表

```
struct Node *p;
p=(struct Node *) malloc(sizeof(struct Node));
p->next=head;
head=p;
```

- 先插入的在链表尾，后插入的在链表头



链表的改造: 虚拟首结点

- ❖ 在链表的任意位置插入与在链表头插入的过程不一样

- 一般插入点的前面是结点，表头的前面是head指针

- ❖ 虚拟首结点：第一个结点不用



- 好处：只需调用一般位置插入即可，无需区分是否是首结点插入

- 链表初始化

```
struct Node head;
```

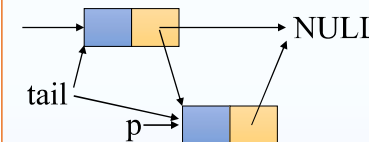


链表的创建

- ❖ 尾插法：在链表尾顺序插入新结点来建立链表

- 插入新结点

```
struct Node *p;
p=(struct Node *) malloc(
    sizeof(struct Node));
p->next=NULL;
tail->next=p;
tail=p;
```

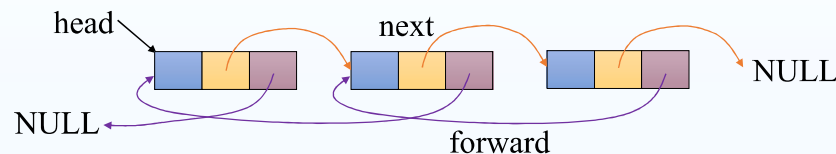


- 先插入的在链表头，后插入的在链表尾

链表的改造: 双向链表

- ❖ 访问链表时，经常会遇到“求上一个结点”的问题

```
struct Node {
    int num;
    struct Node *next, *forward;
};
```

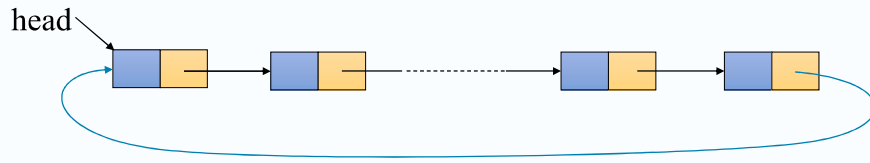


- 优点：节省查询时间；算法更灵活

- 代价：要付出多一倍的指针存储空间

链表的改造: 循环链表

❖ 将链表尾的指针指回链表开始



☞ 优点: 适用于处理需要循环访问的场景

链表小结

❖ 数据结构的存储方式有两种, 根据需求进行选择:

☞ 数组 (顺序存储):

- ❖ 优点: 紧凑连续存储, 可以随机访问, 通过索引快速找到对应元素, 而且相对节约存储空间
- ❖ 缺点: 内存空间必须一次性分配够, 如果要扩容, 需要重新分配一块更大的空间, 再把数据全部复制过去; 而且如果想在数组中间进行插入和删除, 每次必须搬移后面的所有数据以保持连续

☞ 链表 (链式存储):

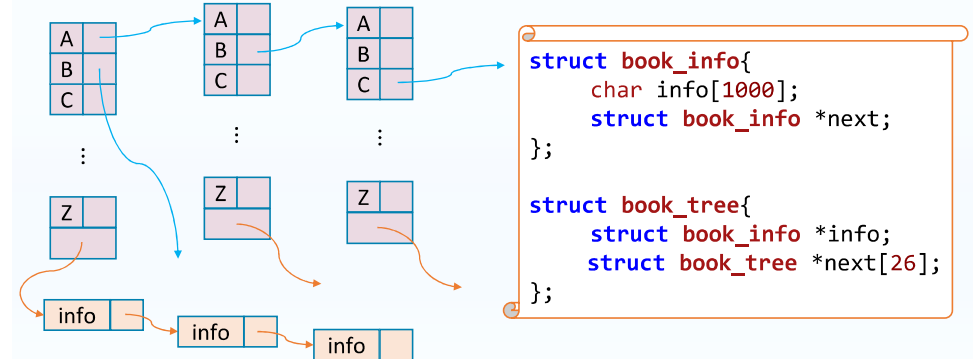
- ❖ 优点: 元素不连续, 不存在数组的扩容问题; 如果知道某一元素的前驱和后驱, 操作指针即可删除该元素或者插入新元素
- ❖ 缺点: 无法根据索引算出对应元素的地址, 不能随机访问; 而且由于每个元素必须存储指向前后元素位置的指针, 会消耗相对多的储存空间

复杂构造数据类型 (例二)

❖ 为一个图书馆的藏书建立一个按书名检索的系统。

☞ 建立一个书目的数据结构

☞ 重名书籍的处理: 将info部分扩展为一个链表



链表小结

❖ 数据结构的基本操作:

- ☞ 初始化、销毁 (内存回收)
- ☞ 数据查询: 个体访问、整体遍历
- ☞ 动态变化: 插入元素、删除元素

❖ 复杂数据处理通常可归结为四个基本操作的组合:

- ☞ 增: `Insert(&L, i, e)` 在第*i*个位置之前插入新的数据元素*e*
- ☞ 删: `Delete(&L, i, &e)` 删除第*i*个数据元素, 并返回值*e*
- ☞ 查: `Traverse(L, visit())` 依次对每个元素调用函数*visit()*
- ☞ 改: `GetElem(L, i, &e)` 用*e*返回*L*中第*i*个数据元素的值

内存调试技巧总结

❖ 内存调试技巧优缺点

技巧	适用场景	优点	缺点
GDB	崩溃定位	实时交互, 查看状态	难发现静默的内存泄漏
Valgrind	深度排查	最彻底, 查得最准	运行速度极慢 (慢 10-50 倍)
ASan	日常开发	速度快, 集成度高	无法检测某些复杂逻辑泄漏
日志记录	长期运行测试	还原分配轨迹	增加代码量和性能开销

❖ 分阶段调试与优化关注点

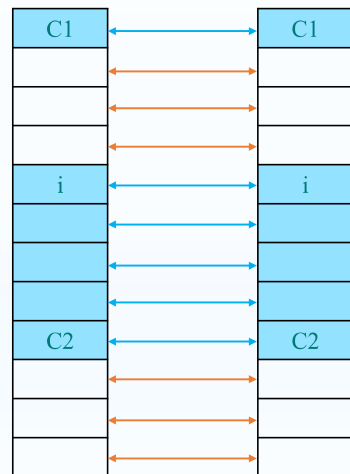
阶段	关注点	核心工具/方法
开发阶段	代码健壮性	静态代码分析 (Cppcheck), 安全函数
测试阶段	捕捉 Bug	Valgrind, ASan, GDB
发布阶段	运行速度	缓存优化, 内存对齐, -O2/-O3 编译优化

数据结构优化：结构体对齐与重排

❖ 结构体中“可能的空洞”的存在，决定了结构体变量间不能进行相等 == 或不等 != 的运算

- ❏ 逐个比较成员是极没有效率的
- ❏ 比较全部字节是一个较好的选择，但是空洞使得比较无法进行

```
struct _a{
    char c1;
    long i;
    char c2;
} a;
```



内存优化概述

- ❖ 内存优化不仅仅是“节省空间”，更核心的目的是提高程序的运行效率（速度）和增强系统的稳定性
- ❖ 内存优化通常从三个维度展开：**数据结构优化**（空间）、**访问模式优化**（速度）和**管理策略优化**（效率）
- ❖ 内存优化不是越极端越好，首先保证逻辑正确，找到性能瓶颈，有针对性的进行优化，过度优化会降低代码的可读性
- ❖ 常见的内存浪费陷阱：优化的最高境界是程序没有优化空间
 - ❏ 过度分配：习惯性地申请 [1024] 长度的内存，其实只用到 10 字节
 - ❏ 重复存储：多个结构体存储了相同的长字符串，可考虑使用常量池
 - ❏ 内存泄漏：虽属于 Bug，但长期运行的泄漏是最大的内存优化敌手

数据结构优化：结构体位域

- ❖ 位段是以位为单位定义长度的结构体类型成员
- ❖ 定义形式：

```
struct 位域结构名 {
    位段成员列表;
};
```

位段成员的说明形式为：
类型说明符 位段名:位段长度;

```
struct packed_data {
    unsigned int a:2;
    unsigned int b:6;
    unsigned int c:4;
    unsigned int d:4;
    unsigned int :0;
    int i;
};
```

位段成员

普通结构体成员

- ❖ 注意：
 - ❏ 一个位段分配在同一个**存储单元**之中，不能跨单元
 - ❖ 可以用 unsigned int:0; 表示从**下一个存储单元**开始存放
 - ❖ 可以用未命名的位段来填充或调整位置，例如 unsigned int:3;

数据结构优化：联合体类型

- 有时为优化内存使用，需要将不同类型变量存储在同一段存储单元中，可以定义**联合体类型**，也叫**共用体类型**

- 任一时刻，只使用其中一个变量
- 本质是以不同方式解读该段内存的数据

- 联合体的定义与用法类似结构，定义格式

```
union 共用体名 {
    类型1 成员名1;
    类型2 成员名2;
    ...
    类型n 成员名n;
};
```

```
union ifcd {
    int i;
    float f;
    char c;
    double d;
};
```

访问模式优化：提高CPU缓存命中率

- CPU 的运算速度极快，而内存相对极慢，为此设计了三级缓存（L1, L2, L3），从内存取数据时，每次加载一个缓存行（通常是 64 字节）

- 空间局部性**：数组在内存中是连续存储的。按行访问二维数组比按列访问快得多，因为按行访问能命中缓存行

```
// 优秀：缓存友好（行优先）
for (int i = 0; i < ROWS; i++)
{
    for (int j = 0; j < COLS; j++)
    {
        sum += matrix[i][j];
    }
}
```

```
// 糟糕：缓存杀手（列优先）
for (int j = 0; j < COLS; j++)
{
    for (int i = 0; i < ROWS; i++)
    {
        sum += matrix[i][j];
    }
}
```

- 减少指针跳转**：链表在内存中是分散的，每次追踪 next 指针都可能导致一次缓存缺失，如果数据量固定，优先使用连续数组代替链表

访问模式优化：内存块的赋值与拷贝

- 循环与memset / memcpy对比总结

特性	循环	memset / memcpy
执行速度	较慢（逐元素）	极快（寄存器级/SIMD 优化）
代码量	较多（3-4 行）	极简（1 行）
灵活性	高（可以处理非连续逻辑）	低（只能处理连续内存块）
类型安全	较好（基于元素类型）	较低（基于字节，需注意类型宽度）
场景建议	逻辑复杂、非 0 初始化	块拷贝、结构体清零

- memset：按字节执行，最适合将内存置零（0）的场景
 - 错误示范**：使用 memset(a, 1, sizeof(a)) 试图把 int 数组全设为 1。由于是按字节填充，最终每个 int 会变成 0x01010101（即 16843009）
- memcpy：第三个参数是总字节数，而不是元素个数
 - 内存重叠**：memcpy 不保证在源地址和目标地址重叠时能正确工作，可能会导致数据损坏；如果发生重叠，必须使用更安全的 memmove

分配策略优化

- 优先使用栈内存**：栈分配只需要移动一次栈指针，而堆分配（malloc）涉及复杂的链表搜索和系统调用

- 优化技巧：小块、生命周期短的数据尽量在栈上分配（函数内）

- 考虑使用内存池**：预先申请大块内存自行管理，避免内存碎片，将多次 malloc 合并为一次，极大提升分配速度

- 延迟分配与重用**

- 延迟分配：直到真正需要使用内存时才调用 malloc
- 对象重用：对于频繁使用的结构体，释放后不要直接 free，而是放入一个“空闲链表”中，下次直接复用

文件操作与数据持久化

吴锋

Email: wufeng02@ustc.edu.cn

文件的基本概念

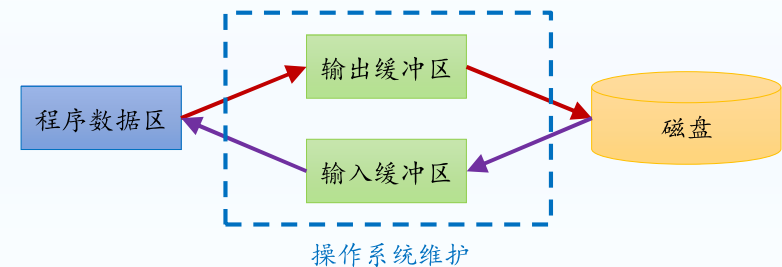
- ❖ 文件是一组相关数据的有序集合，信息最终是以文件形式存储在永久存储设备中（如硬盘、光盘、U盘等）
- ❖ 数据持久性：内存（断电即失） vs 磁盘（永久保存）
- ❖ 操作系统及应用程序往往通过对文件的输入输出完成某项功能；操作系统将与主机相连的输入输出设备也视为文件，如键盘是输入文件，显示器是输出文件
- ❖ 文件按结构形式分为文本文件和二进制文件
 - ❧ 文本文件是全部由字符组成（用ASCII码表示）的具有行列结构的文件，又称为ASCII码文件。便于对字符进行处理，可以直接显示
 - ❧ 二进制文件是按数据在内存中的存储形式存储到文件中，一般不能直接显示
 - ❧ 文本文件的ASCII码也是二进制的，区别在于数值。例：int型整数5在文本文件中保存的是0x35，而在二进制文件中是0x00,0x00,0x00,0x05四个字节

文件的基本概念

- ❖ 文件的读写方式分为顺序存取和随机存取
 - ❧ 顺序存取：从头到尾读写文件所有数据，文件被看成一个字符流，称为流式文件
 - ❖ 网络中所说的流是指一组有序数据序列，代表传输中的数字序列
 - ❖ 向文件顺序写入/读出数据
 - ❧ 打开文件并将文件指针置于文件的开头
 - ❧ 将内存中的数据顺序写入文件/将数据顺序读入内存
 - ❧ 随机存取：随机从文件内指定的位置读写数据
 - ❖ 向文件随机写入/读出数据
 - ❧ 打开文件并将文件定位指针置于待写入/读出的位置
 - ❧ 将内存中的数据写入文件/将数据读入内存
- ❧ 顺序文件只能顺序存取，随机文件两种都可以

文件的基本概念

- ❖ 缓冲文件系统和非缓冲文件系统
 - ❧ 缓冲文件系统是指系统自动地在内存中为每一个正在使用的文件都开辟一个缓冲区
 - ❧ 从内存向磁盘输出数据时，缓冲文件系统先将数据送到内存中的缓冲区，待缓冲区满时才将整个缓冲区的数据一起保存到磁盘文件中（或强制写入）
 - ❧ 从磁盘向内存读入数据时，一般是从磁盘文件中一次读入一批数据到缓冲区，程序从缓冲区获取数据，缓冲区空时再次读入一批数据



文件类型与文件指针

- ❖ 操作系统读写文件时需要的信息
 - ❏ 文件当前的读写位置
 - ❏ 与文件对应的内存缓冲区地址
 - ❏ 文件的操作方式，等
- ❖ 以上信息都存放在“文件信息区”中，一个由系统定义的结构体类型变量，该结构体类型在stdio.h中定义，标识符是FILE，具体定义与编译系统有关
- ❖ C语言标准函数中，系统自动打开和关闭三个标准文件指针
 - ❏ 标准输入（默认绑定键盘）stdin
 - ❏ 标准输出（默认绑定显示器）stdout
 - ❏ 标准出错输出（默认绑定显示器，且无缓冲）stderr

文件的操作模式

- ❖ 文件操作模式 mode

文件操作模式	意义
"r"	以只读方式打开文本文件，该文件应该存在，不存在则报错
"w"	以只写方式建立文本文件，若文件存在则清空文件内容；若文件不存在则建立该文件
"a"	以追加的方式打开文本文件。若文件不存在，则会建立该文件；如果文件存在，则从文件尾开始写（不会清空文件）

- ❏ rb、wb、ab 与 r、w、a 相似，但对象是二进制文件
- ❏ r+、w+、a+ 以读写的操作模式打开文本文件，可切换读写模式
- ❏ rb+、wb+、ab+ 以读写的操作模式打开二进制文件，可切换读写模式

标准输入输出命令行重定向

- ❖ 输入重定向（将stdin / 0 重定向，从文件读取内容代替键盘输入）：
 - ❏ 方法：./my_program < input.txt
 - ❏ 效果：scanf() 等会从 input.txt 文件中读取输入，无需从键盘输入
- ❖ 输出重定向（将stdout / 1 重定向，原本打印在屏幕的内容写入文件）
 - ❏ 方法：./my_program > output.txt （文件如果有内容会被清空）
 - ❏ 追加：./my_program >> output.txt （输出内容补充在文件末尾）
 - ❏ 效果：printf() 等输出结果会保存在 output.txt 文件中，屏幕无显示
- ❖ 错误重定向（将stderr / 2 重定向，仅将错误信息写入文件）：
 - ❏ 方法：./my_program 2> error.log
- ❖ 所有输入输出都重定向到文件（上述重定向可以随意组合）：
 - ❏ 方法：./my_program < input.txt > output.txt 2 > error.log
- ❖ 一个程序的标准输出作为另一个程序的标准输入：random | sum

文件的操作模式（续）

- ❖ 读写模式 (+) 的作用

基础模式	读写模式 (+)	核心区别与应用场景
"r" (只读) 文件必须存在	"r+" (读写) 文件必须存在	多了【修改】的能力。 r: 只能读，不能写。 r+: 可以修改文件中间的任意字节。 场景：修改游戏存档、更新数据库记录。
"w" (只写) 清空/创建	"w+" (读写) 清空/创建	多了【回看】能力。 会清空原有内容，只能读取刚刚写入的数据。 场景：临时文件写完，立刻需要读出来处理。
"a" (追加) 创建/尾部	"a+" (读写) 创建/尾部	多了【读取】能力。 a: 只能向后写。 a+: 可以读取整个文件的历史记录，但写入时强制在末尾添加内容。 场景：聊天软件（读取历史消息 + 发送新消息）。

思考：修改文件中间的某个字符（原地修改），如果使用 w+ 文件会怎么样？

文件指针复用

- ❖ `freopen()` 用于新打开一个文件，直接关联到某个已经打开的文件指针，这样可以复用文件指针，原型定义在头文件 `stdio.h`

`FILE* freopen(char* fname, char* mode, FILE* stream);`

- ❖ `freopen()` 会自动关闭原先已经打开的文件，如果文件指针并没有指向已经打开的文件，则 `freopen()` 等同于 `fopen()`

```
freopen("output.txt", "w", stdout); // stdout 重定向到 output.txt 文件
printf("hello"); // 输出 hello 到 output.txt 文件

freopen("input.txt", "r", stdin); // stdin 重定向到 input.txt 文件
scanf("%d", & i2); // 从 input.txt 文件读取一个整数
```

- ❖ 某些系统允许使用 `freopen()`，改变文件的打开模式。这时，`freopen()` 的第一个参数应该是 `NULL`

```
freopen(NULL, "wb", stdout); //将stdout的打开模式从w改成了wb
```

文本文件的读写

- ❖ 字符串读写函数

`char *fgets(char *str, int n, FILE *stream);`

- ↻ `str`: 存储读取数据的缓冲区，必须确保足够大
- ↻ `n`: 要读取的最大字符数（包括终止符 `\0`），一般为 `sizeof(str)`
- ↻ 返回值: 成功读取一行，返回 `str`；若遇到文件结束或错误，则返回 `NULL`

`int fputs(const char *str, FILE *stream);`

- ↻ 返回值: 成功写入字符串，返回非负值；若出现错误，返回 `EOF`。

- ❖ 示例（通常用于逐行读取或者写入）

```
char buffer[100];
while (fgets(buffer, sizeof(buffer), fp) != NULL) {
    printf("%s", buffer); // 输出读取的行
}
fputs("Hello, World!\n", fp); // 写入字符串
```

文件的读写

- ❖ 文本的文件读写函数

- ↻ 格式化读写函数 `fscanf()` 和 `fprintf()`
- ↻ 字符串读写函数 `fgets()` 和 `fputs()`

- ❖ 二进制文件的读写函数

- ↻ 数据块读写函数 `fread()` 和 `fwrite()`

- ❖ 字节（字符）读写函数

- ↻ 函数 `fgetc()` 和 `fputc()`，同时适用于文本文件和二进制文件的读写
- ↻ 注意：用文本或二进制方式打开文件后，并不限制是用文本还是二进制形式读写数据，但结果会有差别（写值或写ASCII码，读换行符）

二进制文件的读写

- ❖ 数据块（二进制形式）的读写函数

- `size_t fread(void *buf, size_t size, size_t n, FILE *stream);`
- `size_t fwrite(const void *buf, size_t size, size_t n, FILE *stream);`
- ↻ `size_t` 是 `stdio.h` 中定义的类型 (`typedef unsigned int size_t;`)
- ↻ `buf` 为输入/输出在内存中存放的首地址
- ↻ `size` 为读/写的字节数，即数据块的大小
- ↻ `n` 为输入/输出的数据（块）项个数
- ↻ `stream` 为文件指针
- ↻ 返回值: 成功读取/写入的元素数量

二进制文件的读写

❖ 数据块（二进制形式）的读写函数

☞ 调用形式

`fread(buf, size, n, fp);` // 从fp所指文件读入n个大小为size的数据块，存入buf所指内存，返回读取的数据项个数

`fwrite(buf, size, n, fp);` // 从buf所指数据区取size*n个字节写入fp所指文件，返回写到文件中的数据项个数

☞ 示例（用于结构变量的读写）

```
fread(&stu[i], sizeof(struct student), 1, fp);
fwrite(&stu[i], sizeof(struct student), 1, fp);
```

☞ 示例（用于数组的读写）

```
fread(stu, sizeof(struct student), 3, fp);
fwrite(stu, sizeof(struct student), 3, fp);
```

文本文件与二进制文件

❖ 存储整数 12345 的区别：

❖ 使用文本文件：

- ☞ 存储为5个字符 '1','2','3','4','5' (5字节)。
- ☞ 需经过 itoa 转换，解析慢。



❖ 使用二进制文件：

- ☞ 存储为整数原码 0x3039 (4字节)。
- ☞ 内存直接映射，速度极快。



文件的字节读写

❖ 字节读写函数

`int fgetc(FILE *stream);`

☞ 返回值：读取一个字符，返回其ASCII 码；遇到文件结尾或错误，返回 EOF

☞ `fgetc(stdin)` 等效于 `getchar()`

`int fputc(int c, FILE *stream);`

☞ 返回值：成功写入字符，返回字符；若出现错误，返回 EOF

☞ `fputc(c, stdout)` 等效于 `putchar(c)`

❖ 例：

```
fp = fopen("test1.txt", "r");
while ((ch = fgetc(fp)) != EOF) { .... }
```

```
fp = fopen("test1.bin", "rb");
while ((ch = fgetc(fp)), !feof(fp)) { .... }
```

序列化与反序列化

❖ 序列化 (Serialization)：将内存中复杂的数据结构（对象、图、树）转换为线性字节流的过程

- ☞ 空间效率：通常比文本更省空间（浮点数尤其）
- ☞ 时间效率：无须格式解析，适合大数据吞吐
- ☞ 应用场景：数据持久化（存盘）、网络传输（Socket）、进程间通信

❖ 反序列化 (Deserialization)：将字节流还原为内存对象

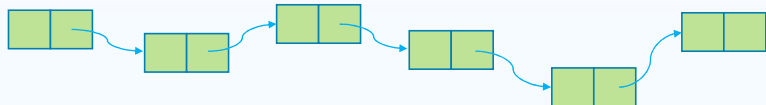


结构体的紧凑对齐

- ❖ 编译器为了提高 CPU 读取内存的效率，通常会在结构体成员之间插入一些的填充字节，如：`struct { char c; int i; }` 大小是8字节，填充3字节
 - ❏ 问题一：如果直接 `fwrite()` 这个结构体，垃圾填充字节也会被写入文件，会导致文件膨胀，浪费磁盘空间
 - ❏ 问题二：如果把这 8 个字节写入文件，然后拷贝到另一台对齐规则不同的机器上读取，数据就会错位
- ❖ 解决方案：`#pragma pack(n)`
 - ❏ 取 `n = 1`，采用紧凑模式，强制结构体按 1 字节对齐（即不留空隙）

链表的序列化与反序列化

- ❖ 链表的每个节点由结构体表示，不仅包含数据域，同时包含指向下一个节点的指针域，如：`struct Node { int data; struct Node *next; }`
- ❖ 序列化时，不能只 `fwrite()` 头节点，必须遍历整个链表，逐个写入：
 1. 写入链表长度 N
 2. 循环 N 次，写入每个节点的数据
- ❖ 反序列化时，先读 N，再循环 N 次 `malloc()` 节点并重新连接



指针与深拷贝

- ❖ 当结构体包含指针成员时，直接 `fwrite()` 的浅拷贝方式，只保存了一个无效的指针地址 (如 `0x7FFF001`) 到文件中
- ❖ 下次读取文件时，该地址已不可用，不能得到信息，还会导致程序崩溃，如：

```
typedef struct {
    char *name;
    int id;
} Student;

Student s = { 101, "Bob" };
// 典型错误写法：直接写入指针（直接 fwrite 整个结构体）
// 后果：写入文件的仅仅字符串在当前内存中的地址，而非字符串本身！
fwrite(&s, sizeof(Student), 1, fp);
```

魔数校验

- ❖ 魔数 (Magic Number) 是位于文件开头的几个固定字节 (比如 `0x46494C45`，对应 ASCII 的 "FILE")，用来快速验证该文件的格式
- ❖ 魔数用于验证输入文件的合法性，防止程序因处理不符合预期格式的文件而崩溃或被恶意利用 (例如病毒程序伪装为图片，欺骗用户点击)
- ❖ 例如，一些常用的文件格式都有魔数：
 - ❏ PDF文件: 开头固定是 `%PDF (0x25 50 44 46)`
 - ❏ Java Class文件: 开头永远是 `0xCAFEFABE` (咖啡宝贝)
 - ❏ PNG 图片: 开头是 `0x89 50 4E 47`
 - ❏ Windows 可执行文件 (EXE): 开头是 `MZ (0x4D 5A)`
- ❖ 序列化时，在二进制文件开头写入魔数，标记文件格式 (序列化协议)
- ❖ 反序列化时，在正式解析二进制文件内容前，先验证魔数并确定文件格式，避免后续代码因格式错误而崩溃

文件定位

- ❖ 位置指针定位到指定位置（随机读写）

`int fseek(FILE *stream, long offset, int origin);`

❏ `origin` 为位置指针的**移动参考点**：

- ❖ 文件头 `SEEK_SET (0)`
 - ❖ 当前位置 `SEEK_CUR (1)`
 - ❖ 文件尾 `SEEK_END (2)`
- ```
for(count=1L; count<=size; count++) {
 fseek(fp, -count, SEEK_END);
 ch = fgetc(fp); // 逆向输出文件的所有字节
}
```

❏ `offset` 为从参考点的**位移量**（字节数、长整型），可以为正值（向文件末尾移动）、负值（向文件开始处移动）或 0（保持不动）

- ❖ 位置指针（重新）定位到开头

`void rewind(FILE *stream);`

❏ 基本等价于 `fseek(stream, 0, SEEK_SET)`，用于将文件指针设置到文件的起始位置，区别是会清除当前文件的错误指示器

## 文件定位

- ❖ 使用 `ftell()` 函数获取文件位置指针的当前位置

`long ftell(FILE *stream);`

❏ 调用成功则返回当前文件指针的位置（以字节为单位），即文件指针相对于文件开头的字节位置；失败时则返回 -1L

❏ `ftell()` 可以跟 `fseek()` 配合使用，先记录内部指针的位置，一系列操作过后，再用 `fseek()` 返回原来的位置

```
long file_pos = ftell(fp); // 记录当前位置
// 一系列文件操作之后，返回之前的位置
fseek(fp, file_pos, SEEK_SET);
```

❏ 还可以先将定位到文件结尾，然后得到文件的总字节数

```
fseek(fp, 0L, SEEK_END); // 将指示器定位到文件结尾
long size = ftell(fp); // 得到文件开始处到结尾的字节数，即文件的大小
```

## 文件定位

- ❖ `fseek()` 函数的调用：

```
fseek(fp, 100L, SEEK_SET); // 从文件头向后（文件尾）移动100个字节
fseek(fp, -10L, SEEK_CUR); // 从当前位置向前（文件头）移动10个字节
fseek(fp, -20L, SEEK_END); // 从文件尾向前（文件头）移动20个字节
```

- ❖ 如以读写模式打开文件（如 `r+`、`w+`、`a+`）

❏ **不改变模式**：仅移动位置指针，不论当前处于读模式还是写模式。

❏ **读写切换规范**：如果在读操作后紧接着进行写操作（或反之），必须在中间插入一个 `fseek` 或 `fflush` 调用，以重置流的读写属性。

❏ **适用场景**：即使 `fseek` 到当前位置（例如 `fseek(fp, 0, SEEK_CUR)`），它也能起到刷新缓冲、允许从读转写或从写转读的作用。

- ❖ 注意：`fseek()` 最好只用来操作二进制文件，不要用来读取文本文件。因为文本文件的字符有不同的编码，某个位置的准确字节位置不容易确定

## 文件定位

- ❖ `fseek()` 和 `ftell()` 有一个潜在的问题，那就是它们都把文件大小限制在 `long int` 类型能表示的范围内（在 32 位计算机上，`long int` 的长度为 4 个字节，能够表示的范围最大只有 4GB）

- ❖ 随着存储设备的容量迅猛增长，文件也越来越大，往往会超出这个范围。因此 C 语言新增了两个处理大文件的新定位函数：`fgetpos()` 和 `fsetpos()`

`int fgetpos(FILE* stream, fpos_t* pos);`

`int fsetpos(FILE* stream, const fpos_t* pos);`

```
fpos_t file_pos;
fgetpos(fp, &file_pos); // 记录当前位置，类似于 file_pos = ftell(fp)

// 一系列文件操作之后，返回之前的位置
fsetpos(fp, &file_pos); // 作用类似于 fseek(fp, file_pos, SEEK_SET)
```

## 文件记录删除

- ❖ 文本文件：必须重写整个文件，去掉那一行（重写策略）
  - ❏ 文本文件被视为一个连续的字节序列，缺乏内部的结构化索引；要在中间删除一行，本质上是在进行字符数组（字符串）的删除操作；由于文件存储在磁盘上，无法像在内存中那样轻易移动后续数据
  - ❏ 因此必须执行复制交换（Copy-and-Swap）策略：读取源文件，过滤掉目标行，写入临时文件，最后替换原子操作；该操作伴随着大量的磁盘 I/O 开销（读写整个文件），这在大数据量场景下是不可接受的性能瓶颈
- ❖ 二进制文件：通常使用“惰性删除”（Lazy Delete）
  - ❏ 在定长的二进制记录中，对数据块拥有**随机访问**的能力，可以通过修改结构体中的状态位，将其标记为‘无效’，因此也称为标记删除法（Flagging）
  - ❏ 此时数据物理上依然存在，但逻辑上已被移除；该操作仅产生极小的 I/O（单次写入），是现代数据库引擎和文件系统底层的通用做法

## 索引文件

- ❖ 如果学生ID不是连续的（如 202301, 202399...）或者结构体大小不一致，无法用数学公式计算 Offset，因此无法实现随机访问
- ❖ 解决方案：建立一个索引表（Index Table）
  - ❏ 主数据文件（Data.bin）：
    - ❖ 乱序存放学生数据，不定长。
    - ❖ 模拟真实环境中数据追加写入的情况。
  - ❏ 索引文件（Index.bin）：只存 {ID, Offset}，定长，按ID排序。
  - ❏ 查询操作：
    - ❖ 先在 Index.bin 二分查找 ID，拿到 Offset；
    - ❖ 在 Data.bin 中直接跳到指定位置读取数据

## 文件记录删除（例）

- ❖ 结构体增加字段：int isDeleted;
- ❖ 删除时：fwrite() 将 isDeleted 设为1
- ❖ 读取时：如果 isDeleted 为 1 则跳过
  - ❏ 优点：删除的速度极快 (O(1))
  - ❏ 副作用：虽然很快，但占用的空间并没有释放。如果删了很多数据，文件会变得很虚大（有效数据很少，文件却很大）
- ❖ 撤销时（Undo）：只需要写一个函数把 isDeleted 改回 0

```
// 定义结构体包含状态标记位
typedef struct {
 ...
 // 0 = 有效 (Active)
 // 1 = 已删除 (Deleted)
 int isDeleted;
} Record;
```

## 文件操作错误

- ❖ 常见错误：文件被占用、磁盘已满、权限不足等。
- ❖ 打开文件时，检查 fopen() 返回值
  - ❏ 如果返回 NULL，说明文件打开失败
- ❖ 读写文件过程中，使用状态检查函数
  - ❏ feof(fp): 检查是否自然读到了文件末尾
  - ❏ ferror(fp): 检查最近一次操作是否发生了读写错误

```
// 循环结束有两种可能：读完了(EOF) 或者 出错了(Error)
if (ferror(fp)) {
 // 硬件/系统错误检测（如磁盘坏道、设备被拔出）
 fprintf(stderr, "读取过程中发生 I/O 错误\n");
} else if (feof(fp)) {
 printf("已安全到达文件末尾 (EOF)。\\n"); // 自然结束检测
}
```

## 错误诊断函数

- ❖ 使用全局变量 `errno` ( `#include <errno.h>` )
  - ⌚ 操作系统维护的一个全局整数，记录了最近一次系统调 用的错误码
  - ⌚ 例如：2 (`ENOENT` - 无此文件) ， 13 (`EACCES` - 无权限)
  - ⌚ 使用 `strerror(errno)` 函数可以将错误码转换为文字描述
- ❖ 使用 `perror()` 函数输出错误信息

```
void perror(const char *str);
```

  - ⌚ 参数：str 为自定义的错误消息前缀，通常是描述错误来源的字符串。它会与 `errno` 中的错误信息一起输出
  - ⌚ 输出：输出的错误信息包括自定义前缀和 `errno` 对应的系统错误描述，二者用冒号分隔。通常输出到标准错误流 (`stderr`)
  - ⌚ 效果相当于 `fprintf(stderr, "%s: %s", str, strerror(errno));`

## 系统临时文件

- ❖ 有时需要使用文件暂存临时数据，用完后需要手动删除，否则大量生成的文件将可能迅速塞满磁盘
- ❖ 解决方案：使用函数 `tmpfile()` 创建临时文件
- ❖ 系统临时文件的特点：
  - ⌚ 在系统临时目录创建文件。
  - ⌚ 程序结束或 `fclose()` 时自动删除，不留垃圾。

```
#include <stdio.h>

FILE *temp = tmpfile();
fprintf(temp, "Temporary Data");
rewind(temp);
// ... 读取使用 ...
fclose(temp); // 文件自动消失
```

## 资源泄露错误

- ❖ 文件句柄是有限资源（涉及文件信息和缓冲区的维护），操作系统限制单进程打开文件数（通常为 512 或 1024）
- ❖ 常见 `fopen()` 的错误
  - ⌚ 在循环中 `fopen()` 却忘记 `fclose()`
  - ⌚ 函数中间遇到错误提前 `return`，跳过了 `fclose()`

```
for (int i=0; i<5000; i++) {
 FILE *fp = fopen("temp.txt", "r");
 // 忘记 fclose(fp)
}
// 运行不久后，fopen 将一直返回 NULL
```

## 文件的删除与重命名

- ❖ 删除指定文件（相当于 Linux 中的 `rm` 命令）

```
int remove(const char* filename);
```

  - ⌚ 接受文件名作为参数。如果删除成功，返回0，否则返回非零值
  - ⌚ 注意：删除文件必须是在文件关闭的状态下。如果是用 `fopen()` 打开的文件，必须先用 `fclose()` 关闭后再删除
- ❖ 重命名文件（相当于 Linux 中的 `mv` 命令）

```
int rename(const char* old_filename, const char* new_filename);
```

  - ⌚ 它接受两个参数，第一个参数是现在的文件名，第二个参数是新的文件名。如果改名成功，返回0，否则返回非零值
  - ⌚ 注意：改名后的文件不能与现有文件同名；另外，如果要改名的文件已经打开了，必须先关闭，然后再改名，对打开的文件进行改名会失败

```
// 通过重命名来移动文件
rename("/tmp/evidence.txt", "/home/user/nothing.txt");
```

## 高级编程与性能优化

吴锋

Email: wufeng02@ustc.edu.cn

## 按位与 (Bitwise AND) &

### ❖ 运算规则

- 0 & 0 = 0;
- 0 & 1 = 0;
- 1 & 0 = 0;
- 1 & 1 = 1;

### ❖ 特殊用法

- 特定位清零
- 保留其它位

```
1010, 1101 (0xAD)
& 0110, 1001 (0x69)

0010, 1001 (0x29)
```

```
xxxx, xxxx
& 0110, 0010 (0x62)

0xx0, 00x0
```

## 位运算符

### ❖ 注意位运算符与逻辑运算区别

| 运算符        | 运算                 | 举例                                        | 优先级从高到低<br>(逻辑非运算符) |
|------------|--------------------|-------------------------------------------|---------------------|
| ~          | 按位取反               | ~flag                                     | (算术运算符)             |
| <<<br>>>   | 左移<br>右移           | a << 2<br>b >> 3                          | (关系运算符)             |
| &<br>^<br> | 按位与<br>按位异或<br>按位或 | flag & 0x37<br>flag ^ 0xC4<br>flag   0x5A | (赋值运算符)             |

## 按位或 (Bitwise Inclusive OR) |

### ❖ 运算规则

- 0 | 0 = 0;
- 0 | 1 = 1;
- 1 | 0 = 1;
- 1 | 1 = 1;

### ❖ 特殊用法

- 特定位置一
- 保留其它位

```
1010, 1101 (0xAD)
0110, 1001 (0x69)
1110, 1101 (0xED)
```

```
xxxx, xxxx
0110, 0010 (0x62)
x11x, xx1x
```



## 按位异或 (Bitwise Exclusive OR) ^

### ❖ 运算规则

⊗  $0 \wedge 0 = 0;$

⊗  $0 \wedge 1 = 1;$

⊗  $1 \wedge 0 = 1;$

⊗  $1 \wedge 1 = 0;$

```
1010, 1101 (0xAD)
^ 0110, 1001 (0x69)

1100, 0100 (0xC4)
```

### ❖ 特殊用法

⊗ 特定位置取反

⊗ 保留其它位

```
xxxx, xxxx
^ 0110, 0010 (0x62)

x~x~x, x~x~x
```

## 按位取反 (Bitwise NOT) ~

### ❖ 运算规则

⊗  $\sim 0 = 1;$

⊗  $\sim 1 = 0;$

```
~ 0110, 1001 (0x69)

1001, 0110 (0x96)
```

## 左移 (Left Shift) <<

### ❖ 运算规则

⊗  $i \ll n$

⊗ 把  $i$  各位全部向左移动  $n$  位, **不会改变  $i$  的值**

⊗ 最左端的  $n$  位被移出丢弃

⊗ 最右端的  $n$  位用 0 补齐

```
5 << 3 = 40
00000101 (0x05) << 3 → 00101000 (0x28)
```

### ❖ 用法

⊗ 若没有溢出, 则左移  $n$  位相当于乘上  $2^n$

⊗ 运算速度比真正的乘法和幂运算快得多

## 右移 (Right Shift) >>

### ❖ 运算规则

⊗  $i \gg n$

⊗ 把  $i$  各位全部向右移动  $n$  位, **不会改变  $i$  的值**

⊗ 最右端的  $n$  位被移出丢弃

⊗ 最左端的  $n$  位用 0 补齐(逻辑右移)

⊗ 或最左端的  $n$  位用符号位补齐(算术右移)

由编译系统的实现者决定。  
为了可移植性, 最好仅对无  
符号数进行移位运算

```
5 >> 2 = 1
00000101 (0x05) >> 2 → 00000001 (0x01)
```

### ❖ 用法

⊗ 右移  $n$  位相当于除以  $2^n$ , 并舍去小数部分

⊗ 运算速度比真正的除法和幂运算快得多

## 位运算的规则

- ❖ 按位运算时，两个操作数长度应相等，
- ❖ 否则先扩展，再运算
  - ☞ 两个操作数右端对齐
  - ☞ 短的数据左端用符号位补齐
    - ❖ 正数或无符号数左端用 0 补满
    - ❖ 负数左端用 1 补满
- ❖ 位运算符都不会改变参与运算变量的值
- ❖ 复合赋值运算符（改变参与运算变量的值）  
 $\&=$ ,  $\wedge=$ ,  $|=$ ,  $\sim=$ ,  $\ll=$ ,  $\gg=$

## 带形参的宏

- ❖ 一般形式：`#define 宏名(形参列表) 宏体`
- ❖ 左圆括号 "(" 必须紧随宏名之后，中间不能有空格，否则就变成不带形参的宏定义（宏体从 "(" 开始）
- ❖ 形参列表中可以出现多个用逗号隔开的不重名的标识符
- ❖ 宏调用的展开：分别用宏调用中的实参文本去替换宏体中对应的形参，宏体中的其它文本不变
  - ☞ 例：`#define AREA(W,H) W*H`  
`AREA(a,b)` 宏展开为 `a*b`  
`AREA(a+1,b+1)` 宏展开为 `a+1*b+1`  
修改为：`#define AREA(W,H) (W)*(H)`  
`AREA(a+1,b+1)` 宏展开为 `(a+1)*(b+1)`
- ❖ 宏展开仅是文本替换，不会进行表达式计算
- ❖ 实参可以为空，但逗号不能省。展开时替换为空串

## 异或运算的特点

- ❖ 异或运算的特点是：
  - ☞ 如果  $a \wedge b = c$ ，那么就有  $c \wedge b = a$  以及  $c \wedge a = b$
- ❖ 此规律可以用来进行最简单的加密和解密以及信息校验
  - ☞ 加密：明文  $\wedge$  密钥 = 密文；解密：密文  $\wedge$  密钥 = 明文
  - ☞ 校验：信息 (a b c d) + 校验码 ( $a \wedge b \wedge c \wedge d$ )
- ❖ 还可以用来交换两位数 a 和 b
  - ☞  $a = a \wedge b$
  - ☞  $b = a \wedge b$ ，即： $(a \wedge b) \wedge b = a$
  - ☞  $a = a \wedge b$ ，即： $(a \wedge b) \wedge a = b$

## 带形参的宏

- ❖ 字符串化操作 `#`：宏展开后，把宏参数变成字符串字面量

```
#define PRINT_VALUE(x) printf("#x " = %d\n", x)

int age = 25;
PRINT_VALUE(age); // 展开为: printf("age " = %d\n", age);
```

- ❖ 连接操作 `##`：宏展开后，把两个符号连接成一个新符号

```
#define CONCAT(a, b) a##b

int value12 = 100;
int result = CONCAT(value, 12); // 展开为: int result = value12;
```

## 条件编译

- ❖ 根据指定的标识符是否未定义过

```
#ifndef 标识符
 程序段1
#endif
```

或

```
#ifndef 标识符
 程序段1
#else
 程序段2
#endif
```

- ❖ 根据常量表达式的值是否非0

```
#if 常量表达式
 程序段1
#endif
```

或

```
#if 常量表达式
 程序段1
#else
 程序段2
#endif
```

或

```
#ifdef 标识符
 程序段1
#elif 常量表达式
 程序段2
#else
 程序段3
#endif
```

## 编译器特性与平台宏

- ❖ 用于条件编译，根据编译器类型或平台调整代码

- ❏ `__GNUC__` : GNU GCC编译器或兼容编译器（如Clang）定义此宏。
- ❏ `__MSC_VER` : Microsoft Visual C++ (MSVC) 编译器定义此宏。
- ❏ `__linux__` : 在Linux操作系统下定义。
- ❏ `_WIN32` / `_WIN64`: 在Windows操作系统下定义。
- ❏ `__cplusplus` : 如果是C++代码文件，则定义此宏。

### 注意事项

- 预处理阶段: 宏定义是在编译之前由预处理器进行简单文本替换的。
- 调试与查看: 可以在GCC中使用 `gcc -dM -E - < /dev/null` 命令查看当前编译器定义的所有宏。
- 不可重定义: 系统预定义宏通常不能使用 `#undef` 重新定义。

```
#ifdef _WIN32
 #include <windows.h>
#else
 #include <unistd.h>
#endif
```

## 核心系统预定义宏 (ANSI C标准)

- ❖ 这些宏在现代C编译器中均可用，通常用于打印调试信息

- ❏ `__FILE__` : 字符串文字，表示当前正在处理的源文件名。
- ❏ `__LINE__` : 整数，表示当前代码行在文件中的行号。
- ❏ `__DATE__` : 字符串文字，表示预处理的日期，格式为 "Mmm dd yyyy"。
- ❏ `__TIME__` : 字符串文字，表示预处理的时间，格式为 "hh:mm:ss"。
- ❏ `__func__` : 字符串，表示当前所在函数的名称（C99标准引入）。
- ❏ `__STDC__` : 如果编译器符合ANSI C标准，则定义为1。

```
#include <stdio.h>
void print_log() {
 printf("File:%s, Line:%d, Function:%s\n", __FILE__, __LINE__, __func__);
}
// 输出示例: File: main.c, Line: 4, Function: print_log
```

## 其它系统预定义宏

- ❖ 整形变量的表示范围: `#include <limits.h>`

- ❏ 字符类型 (`char`) : `CHAR_MIN`, `CHAR_MAX`, `UCHAR_MAX`
- ❏ 整数类型 (`int`) : `INT_MIN`, `INT_MAX`, `UINT_MAX`
- ❏ 长整数类型 (`long`) : `LONG_MIN`, `LONG_MAX`, `ULONG_MAX`

- ❖ 浮点型变量的表示范围: `#include <float.h>`

- ❏ 最大值: `FLT_MAX`, `DBL_MAX`, `LDBL_MAX`
- ❏ 精度 : `FLT_EPSILON`, `DBL_EPSILON`, `LDBL_EPSILON`

## 调试断言中的宏定义

### ❖ 调试断言: #include <assert.h>

❏ void assert(int expr): 如果表达式expr的值为假(即为0),那么它就先向stderr打印一条出错信息,然后通过调用abort来终止程序;否则什么也不做

- ❏ 使用assert时, 频繁的调用会影响程序的性能, 增加额外的开销
- ❏ 在调试结束后, 可以通过插入 #define NDEBUG 来禁用assert调用

```
#ifndef NDEBUG
// 如果NDEBUG的宏在程序中被定义, 则assert定义为空(即什么也不做)
#define assert(e)((void) 0)
#else
// 否则, 如果表达式e为假, 则终止程序, 打印出问题的文件和行号
#define assert(e)\
 ((void)((e) ? ((void) 0) : __assert(#e, __FILE__, __LINE__)))
#endif
```

## 动态库的优缺点

### ❖ 动态库的优点

- ❏ 节省资源: 多个程序可以共享内存中的同一个库副本, 可以实现进程之间的资源共享(因此动态库也称为共享库)
- ❏ 更新方便: 只需替换 .dll 或 .so 文件, 无需重新编译主程序(只要接口不变), 将程序的升级变得简单, 只需要增量更新动态库即可

### ❖ 动态库的缺点

- ❏ 依赖性: 运行环境(当前目录或者系统目录)必须存在对应的库文件, 否则会出现“找不到链接库”的错误
- ❏ 启动略慢: 动态库把对一些库函数的链接载入推迟到程序运行的时期, 运行时需要进行符号重定位, 会有微小的性能开销

## 库的基本概念

### ❖ 在C语言开发中, 库是一种预先编译好的代码集合(二进制文件), 它包含了一系列可复用的函数和变量。根据链接方式分为静态库和动态库

- ❏ 库的作用: 避免“重复造轮子”, 提高编译效率, 保护核心源码
- ❏ 库的本质: 它是 .o (Object file) 文件的打包集合

### ❖ 静态库: 在程序编译链接阶段会被完整地复制到最终的可执行文件中。一旦链接完成, 可执行文件就不再依赖原有的静态库文件

- ❏ 后缀名: Windows 下为 .lib (Library), Linux/Unix/Mac 下为 .a (Archive)

### ❖ 动态库: 在编译时并不直接拷贝到可执行文件中, 而是在程序运行时由操作系统按需加载。可执行文件中仅保留对库的引用(符号表)

- ❏ 后缀名: Windows 下为 .dll (Dynamic Link Library), Linux 下为 .so (Shared Object), Mac 下为 .dylib

## 编程优化概述

### ❖ 优化是一门平衡艺术, 需要在执行速度、内存消耗和代码可读性之间寻找最优解, 对高性能或资源受限的嵌入式开发至关重要

### ❖ 编程中常用的优化方法, 分为四个维度:

- ❏ 数据结构与算法优化: 比任何底层的代码微调都要高效, 例如大规模数据的二分查找比普通的顺序查找性能要高得多
- ❏ 函数级优化: 函数调用的开销虽然微小(入栈、跳转、出栈), 但在高性能计算或高频调用的内层循环中, 这些开销会积少成多
- ❏ 控制指令级优化: 分支和循环是程序逻辑的骨架, 由于CPU的指令流水线和分支预测实现, 不当的编写方式往往会成为性能瓶颈
- ❏ 运算指令级优化: 算术运算指令的执行开销差异巨大, 用低开销的指令组合替代高开销的单条指令

## 函数级优化

- ❖ 核心思路是：减少调用开销、优化数据交换方式、以及辅助编译器决策
- ❖ 调用级：减少函数调用，消除栈帧开销
  - ↪ 使用内联函数 (`inline`) 或者带参数的宏
  - ↪ 尾递归优化 (Tail Recursion)
- ❖ 数据级：减少拷贝、消除内存依赖
  - ↪ 减少参数传递开销 (传值 vs 传址)
  - ↪ 利用 `restrict` 关键字消除别名
- ❖ 编译器级：帮助编译器进行死代码删除或循环外提
  - ↪ 使用 `static` 关键字限制函数作用域及 `const` 关键字优化内存访问
  - ↪ 使用编译器属性告知编译器函数的特殊行为

## 尾递归优化

- ❖ 如果函数的最后动作是调用自身，编译器可以将其转化为 `jmp` 指令，而不是建立新的栈帧，防止栈溢出并提升速度

```
// 普通递归：需要 O(n) 栈空间
int factorial(int n) {
 if (n == 0) return 1;
 return n * factorial(n - 1);
}

// 尾递归：编译器可优化为循环, O(1) 栈空间
int factorial_tail(int n, int accumulator) {
 if (n == 0) return accumulator;
 return factorial_tail(n - 1, n * accumulator);
}
```

## 内联函数

- ❖ 内联函数 (C99)：告诉编译器将函数代码直接“嵌入”到调用点，消除函数调用的压栈、跳转和返回开销
  - ↪ 适用场景：函数体非常小 (如 1-5 行) 且被频繁调用
  - ↪ 与带参数宏的差异：编译器会检查函数的有效性，没有宏的副作用
  - ↪ 注意：`inline` 只是给编译器的建议，并不保证该函数一定会被内联。现代编译器 (如开启 `-O2`) 通常会内联它认为合适的函数

```
// 内联函数
static inline int max(int a, int b) {
 return (a > b) ? a : b;
}

// 带函数的宏, 思考: MAX(a++, b++)会有什么结果?
#define MAX(a, b) (((a) > (b)) ? (a) : (b))
```

## 参数地址传递

- ❖ 函数参数默认是“值传递”。如果传递大型结构体，会发生内存拷贝。改用指针 (地址传递) 只需拷贝一个机器字长 (4 或 8 字节)
- ❖ 传递结构体指针，如不需要修改结构体，可使用 `const` 修饰以保证安全

```
typedef struct {
 double matrix[100][100];
} BigData;

// 不推荐：拷贝 80,000 字节
void process(BigData data);
BigData update_value();

// 推荐：仅拷贝一个 8 字节指针
void process_optimized(const BigData *data);
void update_value_optimized(BigData *data);
```

## 消除指针别名

- ❖ **restrict** 关键字 (C99) 用于修饰指针, 表示在该指针的生命周期内, 它所指向的内存地址不会被其他指针访问, 解决“指针别名”问题
- ❖ 此时, 编译器不需要在每次操作后重新从内存加载数据, 可以直接利用寄存器中的缓存。这是编译器实现 SIMD (向量化) 优化的前提

```
// 告诉编译器 a, b, result 指向的内存绝不重叠
void add_arrays(int *restrict a, int *restrict b, int *restrict result, int n) {
 for (int i = 0; i < n; i++) {
 result[i] = a[i] + b[i]; // 编译器现在可以放心地使用向量化指令
 }
}

int a[10] = {1, 2, 3, 4, 5};
// 错误用法: 让两个 restrict 指针指向了同一个数组重叠的区域
add_arrays(a, a + 1, a, 5); // 错误使用时, 会导致结果不确定
```

## 控制指令级优化

- ❖ **分支语句优化**: 由于现代 CPU 采用分支预测技术, 一旦分支预测失败, CPU 就必须清空流水线并重新加载指令, 这会导致严重的性能损耗
  - ❧ **分支排布**: 排布得当可以显著提升 CPU 分支预测器的命中率
  - ❧ **分支暗示**: 使用内置函数告诉编译器分支的概率, 提高预取成功率
  - ❧ **无分支编程**: 通过其它方式直接计算出结果, 彻底消除分支语句
- ❖ **循环语句优化**: 循环通常占据了程序 80% 以上的执行时间, 优化循环能减少 CPU 的跳转开销, 并为 CPU 提供了更多的指令级并行空间
  - ❧ **循环展开与融合**: 展开或融合小的循环体, 减少判断和跳转开销
  - ❧ **循环外提与交换**: 外提不变量或交换多层循环, 减少冗余计算
  - ❧ **终点判断优化**: 循环判断条件优化, 减少不必要的循环

## 编译器函数属性

- ❖ GCC 和 Clang 提供了一些非标准的属性, 告知编译器函数的特殊行为
  - ❧ **\_\_attribute\_\_((const))**: 告知编译器该函数不读取也不修改任何全局内存, 输出仅依赖于输入参数 (如 `abs()`), 如果多次调用该函数且参数相同, 编译器会将多次调用合并为一次
  - ❧ **\_\_attribute\_\_((pure))**: 告知编译器该函数虽然可能读取全局内存, 但不会修改任何内存 (无副作用), 如果循环中调用该函数且没有修改内存的操作, 编译器可以将调用移出循环
  - ❧ **\_\_attribute\_\_((noreturn))**: 告知编译器该函数执行后不会返回 (如 `exit()` 或死循环), 编译器可以优化调用后的控制流, 减少不必要的清理代码

```
int square(int x) __attribute__((const));

for (int i = 0; i < 100; i++) {
 // 编译器发现 square(10) 结果不变, 只会计算一次, 不会重复调用
 sum += square(10);
}
```

## 分支排布优化 (二)

- ❖ **先过滤“少数派”**: 当逻辑中包含异常处理或边界检查, 使用卫语句 (Guard Clauses) 提前退出, 而不是将核心逻辑嵌套在庞大的 `if` 块中

```
// 不推荐: 深层嵌套
void process(data) {
 if (data != NULL) {
 if (is_ready(data)) {
 // 核心逻辑...
 }
 }
}

// 推荐: 卫语句, 快速失败
void process(data) {
 if (data == NULL) return; // 异常情况提前退出
 if (!is_ready(data)) return; // 异常情况提前退出
 // 核心逻辑保持在函数主层级
}
```



## 分支排布优化（三）

### ❖ 逻辑短路的布局：

- ❏ && (与)：将最可能为假或计算开销最小的表达式放在左侧
- ❏ || (或)：将最可能为真或计算开销最小的表达式放在左侧

### ❖ switch-case 的排布：

- ❏ 紧凑数值：尽量让 case 的值连续（如 1, 2, 3），编译器会生成跳转表 (Jump Table)，查找效率极高 O(1)，与分支排布顺序无关
- ❏ 离散数值：如果值很分散（如 1, 100, 5000），编译器不会生成跳转表，而会将其转为类似二分查找树，将它们集中在数值较近的范围内会有所帮助
  - ❖ 手动分组：如果分散数值有明显的“集群”特征，可以通过一个简单的 if 分组判断，人为减少了每个 switch 内部的搜索深度分组
  - ❖ 热点优先：如果知道某个值（比如 1）占了大量的调用频率，那么把它留在 switch 树里是不划算的，可以通过一个 if 判断移出 switch

## 无分支编程（二）

- ❖ 使用三元运算符 (a ? b : c)：在许多现代编译器中，三元运算符 会被编译成 cmov（条件移动）指令，不涉及分支预测和 jmp（跳转）指令，因此在处理逻辑简单但结果随机的情况时，效率更高

```
// 将数值限制在 [min, max] 区间内
if (x < min) x = min; else if (x > max) x = max; // 传统做法：使用分支语句
x = (x < min) ? min : ((x > max) ? max : x); // 无分支版：使用三元运算符
```

- ❖ 利用算术运算、位运算或逻辑运算来消除 if 语句，在处理大规模随机数据时效率极高，可以避免 CPU 分支预测失败导致的指令流水线清空

```
// 大规模排序算法中整数值的条件交换
if (a > b) { int temp = a; a = b; b = temp; } // 传统做法：使用分支语句

// 无分支版：使用位运算
int mask = -(a > b); // 如果 a > b, mask 为全 1
int t = (a ^ b) & mask; a ^= t; b ^= t;
```

## 无分支编程（一）

- ❖ 使用查表法 (Lookup Table) 代替分支：如果分支的作用只是根据一个输入值映射到一个输出值，那么数组查表是最高效的方案

```
// 优化前：冗长的 switch
int get_days(int month) {
 switch(month) {
 case 1: case 3: case 5: return 31;
 case 4: case 6: return 30;
 // ... 其他 case
 }
}

// 优化后：查表法
static const int days_table[] = {0, 31, 28, 31, 30, 31, 30,
31, 31, 30, 31, 30, 31};
int get_days_fast(int month) {
 return days_table[month]; // O(1) 时间复杂度, 零分支
}
```

## 循环展开与融合

- ❖ 循环展开：通过减少循环次数，增加单次循环内的操作量，从而减少循环计数和条件跳转的次数
- ❖ 循环融合：如果两个循环遍历相同的范围，合并它们可以减少循环控制开销，并提高数据重用率

```
// 展开前
for (int i = 0; i < 1000; i++) {
 sum += data[i];
}

// 展开后：减少了 75% 的循环开销
for (int i = 0; i < 1000; i += 4) {
 sum += data[i];
 sum += data[i+1];
 sum += data[i+2];
 sum += data[i+3];
}
```

```
// 融合前：遍历两次
for (int i=0; i<n; i++) {
 a[i] = b[i] + 1;
}
for (int i=0; i<n; i++) {
 c[i] = a[i] * 2;
}

// 融合后：遍历一次
for (int i=0; i<n; i++) {
 a[i] = b[i] + 1;
 c[i] = a[i] * 2;
}
```

## 循环外提

- ❖ 循环外提：将循环中不随循环变量变化的代码（常量计算、重复的指针寻址、函数调用等）移到循环外面，减少冗余计算

```
// 外提前：每次循环都在重复计算
for (int i = 0; i < n; i++)
 data[i] = i * (a + b) / scale;
```

```
// 外提后：提炼出常量因子
double factor = (a + b) / scale;
for (int i = 0; i < n; i++)
 data[i] = i * factor;
```

```
// 外提前：每次循环都要写入内存
for (int i = 0; i < n; i++)
 *res += arr[i];
```

```
// 外提后：只写一次内存
int temp = 0; // 使用寄存器暂存
for (int i = 0; i < n; i++)
 temp += arr[i];
*res = temp; // 最后一次性写回内存
```

```
int my_strchr(char * s, char c) { // 思考：该实现有什么问题？该如何优化？
 for (int i = 0; i < strlen(s) - 1; i++)
 if (s[i] == c) return i; // 看s中是否包含 c
 return -1;
}
```

## 终点判断优化（一）

- ❖ 将循环判断条件优化为与0比较，大多数CPU都有jz指令来检查计算结果是否为0，比两个非零变量的比较（i < n）少一条cmp指令

```
// 传统做法：需要执行 i < n 的比较操作
for (int i = 0; i < n; i++) {
 process(data[i]);
}
```

```
// 优化做法：如果顺序不重要，递减到 0
for (int i = n; i-- > 0;) {
 process(data[i]);
}
```

## 循环交换

- ❖ 循环交换：在嵌套循环中，改变内外循环的顺序，最忙的循环放最里面，并确保内存访问是连续的，提高缓存命中率

```
// 交换前：内存跳跃极大, Cache Missing 严重
for (int column = 0; column < 1000; column++) {
 for (int row = 0; row < 5; row++) {
 sum += table[row][column];
 }
}
```

```
// 交换后：连续访问内存, 速度提升可达 10 倍以上
for (int row = 0; row < 5; row++) {
 for (int column = 0; column < 1000; column++) {
 sum += table[row][column];
 }
}
```

## 终点判断优化（二）

- ❖ 哨兵位技术：在数组搜索等逻辑中，通过在数组末尾放置一个“哨兵”，将“越界检查”和“值检查”合并为一个检查，减少了大量的分支判断

```
// 传统做法：当 n 非常大时，每次都要判断 i < n 和 是否找到目标
int find(int *arr, int n, int target) {
 for (int i = 0; i < n; i++)
 if (arr[i] == target) return i;
 return -1;
}
```

```
// 优化做法：哨兵位（假设数组空间足够）
int find_optimized(int *arr, int n, int target) {
 int last = arr[n-1]; arr[n-1] = target; // 设置哨兵
 for (int i = 0; arr[i] != target; i++); // 循环内只需一次判断
 arr[n-1] = last; // 恢复数据
 if (i < n - 1 || last == target) return i; else return -1;
}
```

## 运算指令级优化

- ❖ 不同的算术运算对 CPU 产生的开销差异极大，通常情况下，加法、减法、位运算只需要 1 个 CPU 时钟周期，而乘法可能需要 3-10 个周期，除法和求余则可能需要 20-80 个周期
- ❖ 指令级优化的核心思想是：用低开销的指令组合替代高开销的单条指令
  - ⌚ 极低开销运算（通常1个机器周期或更少）：位运算、赋值（=）、加减法
  - ⌚ 低开销运算（1-数个机器周期）：乘法（\*）、逻辑运算（&&、||）
  - ⌚ 中开销运算（数十个机器周期）：除法与取模（/, %）、分支判断（if-else）
  - ⌚ 高开销运算（函数调用及内存访问开销）：sin()、cos()、log()、pow()
  - ⌚ 极高开销运算（动态内存分配与文件操作）：malloc()、printf()、fread()

位运算 < 整数加减 < 乘法 < 除法 < 函数调用 < 内存读取 < 系统调用

## 代数恒等式简化

- ❖ 避开平方根：在比较长度时，比较平方值

```
// 优化前
if (sqrt(x*x + y*y) > radius) { ... }
// 优化后：消除昂贵的 sqrt()
if ((x*x + y*y) > (radius * radius)) { ... }
```

- ❖ 合并公共项

```
// 优化前
y = a * b * c + a * b * d;
// 优化后
y = (a * b) * (c + d); // 减少了一次乘法
```

- ❖ 定点数运算代替浮点数

```
// 处理货币,不使用 float 1.25 元,使用 int 表示"分"
// 在没有浮点运算单元的设备中,float 运算由软件库模拟,速度极慢
int price_in_cents = 125;
int total = price_in_cents * quantity;
```